Министерство науки и высшего образования Российской Федерации
Федеральное государственное автономное образовательное
учреждение высшего образования
«Московский государственный технический университет
имени Н.Э. Баумана
(национальный исследовательский университет)»
(МГТУ им. Н.Э. Баумана)

ФАКУЛЬТЕТ Информатика и системы управления

КАФЕДРА Системы обработки информации и управления

РАСЧЕТНО-ПОЯСНИТЕЛЬНАЯ ЗАПИСКА

К ВЫПУСКНОЙ КВАЛИФИКАЦИОННОЙ РАБОТЕ

НА ТЕМУ:
Разработка автоматизированной научно-технической библиотеки с расширенной системой поиска

Студент ИУ5-45МВ _____ Л.В. Литвинович
(Группа) (Подпись, дата) (И.О. Фамилия)

Руководитель _____ М.В.Виноградова
(Подпись, дата) (И.О. Фамилия)

Нормконтролер _____ Ю.Н.Кротов
(Подпись, дата) (И.О. Фамилия)

2025 г.

АННОТАЦИЯ

В настоящее время автоматизированные научно-технические библиотеки становятся лабораториями знания. В связи с этим изменяются требования к их функционалу. В данной работе предлагаются новые пути повышения качества традиционного поиска по метаданным документов, а также предлагаются другие виды поиска; поиск по пространству признаков документов и расширенный поиск через интеграцию с информационными платформами.

Основная функция библиотек – это быстрый и релевантный поиск документов. Опыт библиотек в поиске документов по метаданным может удачно использоваться в малом бизнесе при решении задач скорости выполнения запросов к базам данных с большим количеством записей. Этот опыт дает возможность отдалить момент горизонтального масштабирования баз данных, который связан с большими финансовыми затратами и риском потери данных. В нашей работе были исследованы факторы снижения скорости сложных запросов к таблицам и коллекциям с большим количеством данных. Также для расширения возможностей поиска мы рассмотрели возможность интеграции с информационной платформой arXiv. Этот вид поиска раскрывает перед пользователем возможность получения документов по запрашиваемой тематике сразу в большом количестве. Кроме того, научные статьи отсортированы по дате и пользователь получает самые свежие статьи. В отличие от традиционного поиска конкретные метаданные документа часто возвращают ответ, что источник не найден по причине, что его нет в базе данных. Также такой поиск может выдавать нерелевантные статьи, если в запросе пользователя присутствуют аббревиатуры, часто встречающиеся в других областях знания. Этот поиск хорошо работает при обозначении в запросе темы.

Также в нашей работе предлагается новый вид поиска по векторам признаков документов с помощью предобученной модели SBERT. Этот поиск решает проблемы первых двух видов поиска. Так при вводе ошибочного

символа или слова, запрос будет выполнен и возвратит пользователю пять релевантных документов. Сильной стороной поиска по пространству признаков является понимание семантического смысла моделью; связывая тексты по семантической схожести этот поиск наиболее устойчив к ошибкам, связанным с разным смыслом одинаковых аббревиатур. Однако, и у этого поиска есть трудности. Документы, в которых затрагиваются смыслы из смежных областей знания, могут быть классифицированы моделью либо к одному классу знаний, либо к другому. Такая неоднозначная классификация документов, а равнозначно и запросов пользователя, решается тренировкой модели на узкоспециализированных областях знаний. Таким образом, в данной работе разрабатывается автоматизированная научно-техническая библиотека с расширенным и улучшенным функционалом.

СОДЕРЖАНИЕ

АННОТАЦИЯ 2
СПИСОК СОКРАЩЕНИЙ И ОБОЗНАЧЕНИЙ 6
ВВЕДЕНИЕ 11
1 ВОЗМОЖНОСТИ АВТОМАТИЗИРОВАННЫХ НАУЧНО-ТЕХНИЧЕСКИХ БИБЛИОТЕК В СИСТЕМЕ ПОИСКА ИНФОРМАЦИИ 12
1.1 Научно-технические библиотеки и доступ к научно-технической информации 13
1.2 Движение за открытый доступ. Платформа arXiv 15
1.3 Время выполнения запросов 20
1.4 Поиск по пространству признаков литературного источника 20
2 ИССЛЕДОВАНИЕ И РАЗРАБОТКА АВТОМАТИЗИРОВАННОЙ СИСТЕМЫ НАУЧНО-ТЕХНИЧЕСКОЙ БИБЛИОТЕКИ 22
2.1 Этапы проектирования 22
2.2 Исследование возможностей интеграции с бесплатной информационной системой arXiv 38
2.3 Исследование возможностей повысить скорость выполнения запросов на чтение 40
2.3.1 Информационная модель сложного SQL-запроса с применением B-tree индексов, как объект теоретического исследования 43
2.3.2 Логические схемы выбора оптимизатором плана выполнения запроса 46
2.3.3 Исследование факторов оптимизации сложных запросов к таблицам/коллекциям с большим количеством записей 48
2.4 Исследование возможности предоставления пользователю нового вида поиска, основанный на векторном пространстве признаков литературного источника 50

3 РЕЗУЛЬТАТЫ ИССЛЕДОВАНИЯ И РАЗРАБОТКИ АВТОМАТИЗИРОВАННОЙ СИСТЕМЫ НАУЧНО-ТЕХНИЧЕСКОЙ БИБЛИОТЕКИ 56
3.1 Исследование факторов снижения скорости запросов пользователя 57
3.1.1 Исходные данные 57
3.1.2 Методы 58
3.1.3 Результаты исследования и их обсуждение 59
3.2 Исследование возможностей семантического поиска в автоматизированной научно-технической библиотеке 66
3.2.1 Сбор и подготовка тематических данных 67
3.2.2 Процесс дообучения модели 67
3.2.3 Анализ результатов 67
ЗАКЛЮЧЕНИЕ 72
СПИСОК ИСПОЛЬЗОВАННЫХ ИСТОЧНИКОВ 74
ПРИЛОЖЕНИЕ А 79

СПИСОК СОКРАЩЕНИЙ И ОБОЗНАЧЕНИЙ

БД — база данных;

СУБД — система управления базами данных;

АНТБ — автоматизированная научно-техническая библиотека;

Информационная модель — совокупность информации, характеризующая существенные свойства и состояния объекта, процесса, явления, а также взаимосвязь с внешним миром;

Оптимизатор запросов — выбор данных из базы, движок СУБД, который пытается наилучшим (наиболее быстрым, менее затратным) образом использовать ресурсы. В качестве результата оптимизатор выдает «План выполнения запроса»;

B-tree — это структура данных, которая предоставляет отсортированные данные и позволяет выполнять поиск, последовательный доступ, вложения и удаления в отсортированном порядке. B-tree обладает высокой способностью хранить системы, которые записывают большие блоки данных. B-tree упрощает двоичное дерево поиска, допуская узлы с более чем двумя дочерними элементами; одна метастраница хранится в фиксированной позиции в начале первого сегментного файла индекса, все остальные страницы являются либо конечными страницами, либо внутренними страницами;

B+tree — это структура данных, в которой внутренние узлы в индексе не имеют ссылок на страницы документа, но содержат только ссылки на соседние узлы и на узлы уровнем ниже; только листовые узлы B+tree имеют информацию страниц документа. Такая особенность и позволяет сегментировать данные без потерь;

Составные индексы — индексы в MongoDB, которые состоят из нескольких полей; порядок полей в составном индексе имеет значение. Например, если составной индекс состоит из { author: 1, name : -1 }; индекс

сортируется сначала по полю author и потом в пределах поля author по поля name;

Покрывающие индексы —индексы, в выражении которых присутствуют все поля, используемые в запросе. Оптимизатору не надо обращаться к таблице в памяти, он получает информацию из таблицы индекса;

Селективность —количество строк в таблице, удовлетворяющих запросу, относительно к общему количеству строк в наборе; чем меньше строк для поиска, тем выше селективность;

Эргономичность запроса —время выполнения запроса (производительность СУБД). При запросах к большим данным выбор оптимизатором плана сканирования индекса имеет огромное значение потому, что отображается во времени ожидания ответа для пользователя. Создание условий, в которых человек может работать комфортно и эффективно относится к области знаний эргономики, поэтому под эргономичностью запроса мы будем понимать время его выполнения;

Эргономические показатели запроса —селективность полей в индексе B-tree, учет NULL значений в полях индекса B-tree, покрывающие индексы B-tree;

ЧО — человек-оператор, осуществляющий трудовую деятельность, основу которой составляет взаимодействие с объектом воздействия, машиной и средой на рабочем месте при использовании информационной модели и органов управления [1];

ЗА — время восприятия зрительным анализатором человека светового сигнала [1];

Суперкласс — это компоненты наследования данных, в которых определенные атрибуты и характеристики сущности наследуются от родительского объекта к дочерним объектам или сущностям.

Эмбеддинг (embedding) — числовые представления текста, позволяющие измерять семантическое сходство;

BERT (Bidirectional Encoder Representations from Transformers) – языковая модель на основе архитектуры трансформеров, обученная учитывать контекст слова с обеих сторон (двунаправленная), позволяет создавать эмбеддинги, используется в классификации, поиске, распознавании именованных сущностей;

SBERT (Sentence Bidirectional Encoder Representations from Transformers) – вариант модели BERT позволяет эффективно вычислять семантическое сходство между предложениями с высокой скоростью и точностью, значительно ускоряя поиск и кластеризацию по сравнению с обычным BERT;

SBERT_all-mpnet-base-v2 (Masked and Permuted Pre-training for Language Understanding) – модель SBERT, отличие от классического BERT, который маскирует отдельные слова и обучается их предсказанию, переставляет порядок токенов, что помогает модели лучше улавливать взаимосвязи между словами и контекстом.

DOI (Digital Object Identifier) – цифровой идентификатор объекта литературного источника. Стандарт ISO 26324:2012 регулирует структуру и использование DOI;

ISBN (International Standard Book Number) — международный стандартный книжный номер, присваиваемый изданиям книг и других печатных публикаций для их однозначной идентификации в книгоиздательстве и библиотечном деле;

ISSN (International Standard Serial Number) — международный стандартный номер сериального издания (журнал, газета, серийная публикация);

SICI –стандартный код, предназначенный для идентификации статей в журнале. Включает информацию о журнале, томе, номере, страницах и дате публикации;

УДК — международная система классификации знаний в библиотеках и информационных системах. Позволяет классифицировать документы по тематике с помощью цифровых кодов.

ПО — программное обеспечение;

AI (Artificial Intelligence) — искусственный интеллект;

HTTP — протокол передачи гипертекста, используемый для обмена информацией в сети Интернет;

ЦП —центральный процессор.

GiN — индекс, используемый в БД для текстового поиска или индексации массивов;

GiST — специализированные индексы для различных типов данных и запросов, таких как пространственные или полнотекстовые запросы;

TID (Tuple Identifier) — это уникальный идентификатор строки (кортежа) в таблице базы данных, который указывает номер блока (страницы) и индекс кортежа внутри этого блока;

MSE (Mean Squared Error) — это метрика качества модели, измеряющая среднее значение квадратов разностей между предсказанными и фактическими значениями;

NLP (New Lean Product development) — область искусственного интеллекта, занимающаяся взаимодействием компьютеров и человеческих языков. Включает задачи анализа, понимания, генерации и перевода текстов и речи;

Accuracy — доля правильно классифицированных объектов от общего числа объектов. Используется для оценки качества классификационных моделей.

Precision —доля правильно предсказанных положительных объектов от всех объектов, отнесённых моделью к положительным.

Recall —доля правильно предсказанных положительных объектов от всех фактических положительных объектов.

F1-мера —это метрика оценки качества классификатора, представляющая собой гармоническое среднее между точностью (Precision) и полнотой (Recall).

ВВЕДЕНИЕ

Проблеме скорости выполнения SQL-запросов посвящены работы ряда авторов [1,2,3,4,9,10,11,12,13,14,15,16]. На время выполнения запросов влияет множество факторов: индексы, объем данных, выбор плана выполнения запроса оптимизатором, текущая нагрузка на базу данных, задержки на уровне сети.

Необходимость оптимизации запросов заключается также в снижении нагрузки на СУБД. От структуры составленного запроса зависит, какое количество вычислительных ресурсов (оперативная память, процессорное время) будет использоваться.

В данной работе разрабатываются информационные модели сложных поисковых запросов с B-tree индексами на примерах PostgreSQL и MongoDB. В соответствии с контекстом данных онлайн научной библиотеки предполагалось использование документо-ориентированной MongoDB. Выбор PostgreSQL объясняется лидерством этой СУБД перед другими при запросах к большим данным на чтение [3]. В обеих СУБД используется B-tree структура данных, в MongoDB используется B+tree. Данный факт также учитывался при выборе MongoDB в нашем эксперименте. Структура данных обеих СУБД позволяет использовать запросы с B-tree индексами. Запросы с B-tree индексами по умолчанию возвращают упорядоченные данные. Это значительно сокращает время выполнения запросов. Поэтому, мы выбрали исследование сложных запросов с применением B-tree индексов.

1 ВОЗМОЖНОСТИ АВТОМАТИЗИРОВАННЫХ НАУЧНО-ТЕХНИЧЕСКИХ БИБЛИОТЕК В СИСТЕМЕ ПОИСКА ИНФОРМАЦИИ

Приступая к разработке автоматизированной системы необходимо учитывать особенности архитектуры этой системы. Подход, основанный на вариантах использования будущей автоматизированной системы [1] представляется наиболее рациональным. Например, модель «менеджер» предусматривает параллельную работу подсистем, в «широковещательной модели» каждая подсистема уведомляет обработчик об интересе к тому или иному событию. «Модель управления» исключает параллельную работу подсистем, системный контроллер руководит работой остальных систем; например, запрос клиента обрабатывается по схеме вызов-возврат.

Разработка АНТБ предполагает выполнение определенных функциональных требований. Возможно классифицировать функциональные требования к АНТБ по направлениям:

1) интерфейс взаимодействия с пользователем;

2) обеспечение интеграции и безопасности данных;

3) обеспечение производительности и масштабируемости;

4) управление данными;

5) обеспечение выполнения основных требований обслуживания.

Функциональные требования определяют выбор архитектуры АНТБ.

Возможно выделить виды архитектуры для АНТБ:

1) клиент-серверная;

2) многоуровневая;

3) компонентная;

4) каркасы.

Проектирование АНТБ ориентировано на следующие этапы:

1) определение узлов и сетевых конфигураций;

2) определение подсистем (прикладных подсистем, сервисных подсистем);

3) определение интерфейсов подсистем;

4) определение архитектурно-значимых классов проектирования;

5) определение обобщенных механизмов проектирования.

Также для чистоты архитектуры используют структурные паттерны. Например, «Абстрактная фабрика», «Декоратор», «Фасад», «Строитель» и другие. Так паттерн «Фасад» используется для управления базами данных, для поиска и доступа к различным типам научных публикаций в клиент-серверной архитектуре ИРБИС, МегаПро, eLIBRARY.RU. Отметим преимущества «Фасад»:

1) предоставление пользователям единого интерфейса взаимодействия с системой;

2) облегчает поддержку и развитие системы, скрывая сложный код.

К недостаткам паттерна «Фасад» можно отнести зависимость от интерфейса компонентов, использующих его.

1.1 Научно-технические библиотеки и доступ к научно-технической информации

Научно-технические библиотеки обеспечивают доступ к научно-технической информации для широкого круга пользователей. В таблице 1 приведены примеры нескольких наиболее популярных научно-технических библиотек.

Таблица 1 — Примеры услуг лучших научно-технических библиотек

Название	Тип ресурса	Тематика	Доступ
Российская Государственная Библиотека	электронная библиотека	разнообразная	через подписку
eLibrary.ru	научные статьи, журналы и книги	разнообразная	регистрации для доступа к полным текстам

Название	Тип ресурса	Тематика	Доступ
CyberLeninka	полнотекстовая база данных статей	социальные, естественные, инженерные и гуманитарные науки	бесплатный доступ к статьям
Google Scholar	Реферативно-библиографический	разнообразная	бесплатный доступ

Результаты анализа предоставляемых услуг в других библиотеках [2-11] представлены в таблице 2. Более подробные материалы представлены в приложении[А1, А2, А3, А4, А5, А6, А7].

Таблица 2 — Результаты анализа предоставляемых услуг в других библиотеках

Показатели	Б1	Б2	Б3	Б4	Б5	Б6	Б7	Б8	Б9	Б10
Имеет собственную библиотеку	-	-	+	-	-	+	-	-	-	-
Бесплатный доступ	-	-	+	+	-	+	-	-	+	-
Метрики отслеживания	-	+	+	+	+	-	+	+	+	+
Скорость поиска	-	+/-	-	-	+/-	+	+	+	+	+
Скорость скачивания	-	+/-	-	-	+/-	+	+/-	+/-	+/-	+/-
Подгрузка книги по одной странице	-	+	+	+	+	+	+	+	+	+
Возможность делать закладки	-	+/-	+	-	+/-	+/-	+	+	+	+
Возможность комментариев	-	-	+	-	-	+/-	-	-	-	-

Сравнивая результаты обеих таблиц можно сделать вывод о том, что в крупных библиотечных системах предоставлен бесплатный доступ к литературным источникам. Также можно отметить, что у некоторых библиотек:

1) нет открытого доступа к ресурсам (20% сайтов);

2) присутствуют метрики отслеживания для сбора личных данных пользователя.

3) отмечается низкая скорость выполнения запросов при чтении источника из-за того, что сайт подгружает по одной странице только во время перелистывания (30% сайтов);

Таким образом, выявляются проблемы научно-технических библиотек в предоставлении доступа к литературным источникам по запросам пользователей:

1) ограниченность доступа;

2) низкая скорость выполнения запросов;

3) необходимость интеграции с другими информационными системами и платформами.

Для решения этих проблем выполняется данная работа.

1.2 Движение за открытый доступ. Платформа arXiv

Платформа arXiv создана в 1991 году вначале движения за открытый доступ. arXiv открыл возможности для исследователей обмениваться публикациями в открытом доступе. Сразу после публикации статьи доступны для всех и это ускоряет развитие научного познания. В поддержку принципов открытого доступа Европейский исследовательский совет в 2013 году объявил о сотрудничестве с arXiv. Статьи, опубликованные в arXiv индексируются в Web of Science, Google, Scopus.

Возможности лабораторий arXiv охватывают широкий спектр областей знаний, представлены на рисунках 1 и 2.

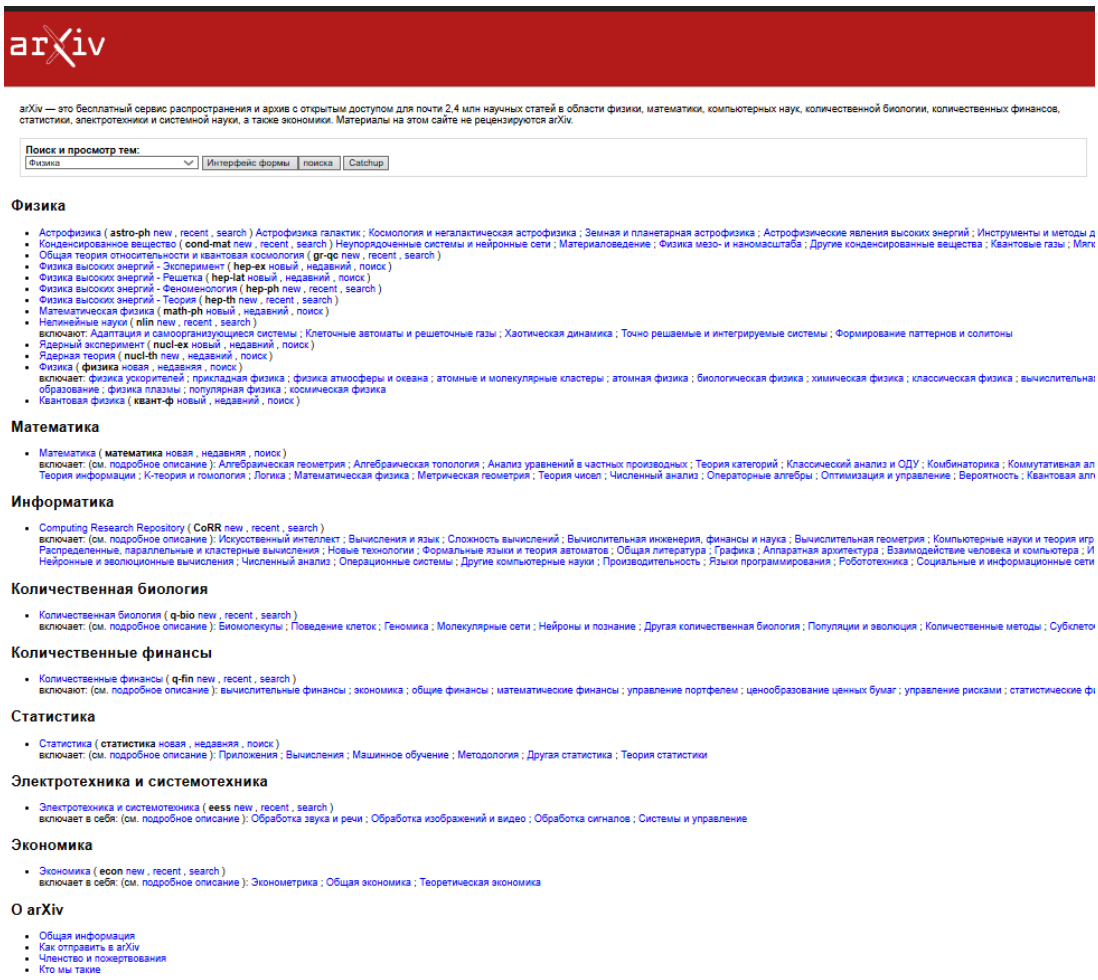


Рисунок 1 — Возможности лабораторий arXiv охватывают широкий спектр областей знаний



Рисунок 2 — Интерфейс лаборатории Hugging face Spaces

Существует возможность получения ресурса и проведения своего исследования на готовых моделях. Пример пользовательского интерфейса на работе одной лаборатории arXiv с возможностью выбора моделей для исследования представлен на рисунке 3.

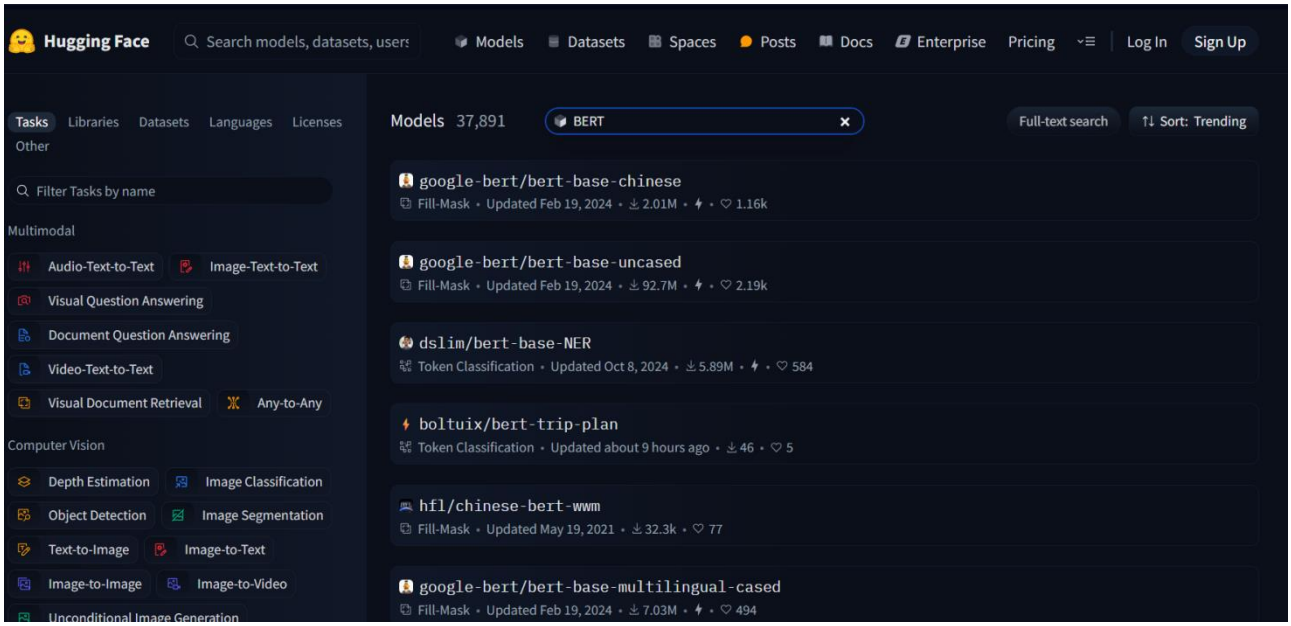


Рисунок 3 — Возможность выбора моделей, набора данных, методов исследования

Возможность выбора набора данных для исследования представлена на рисунке 4.

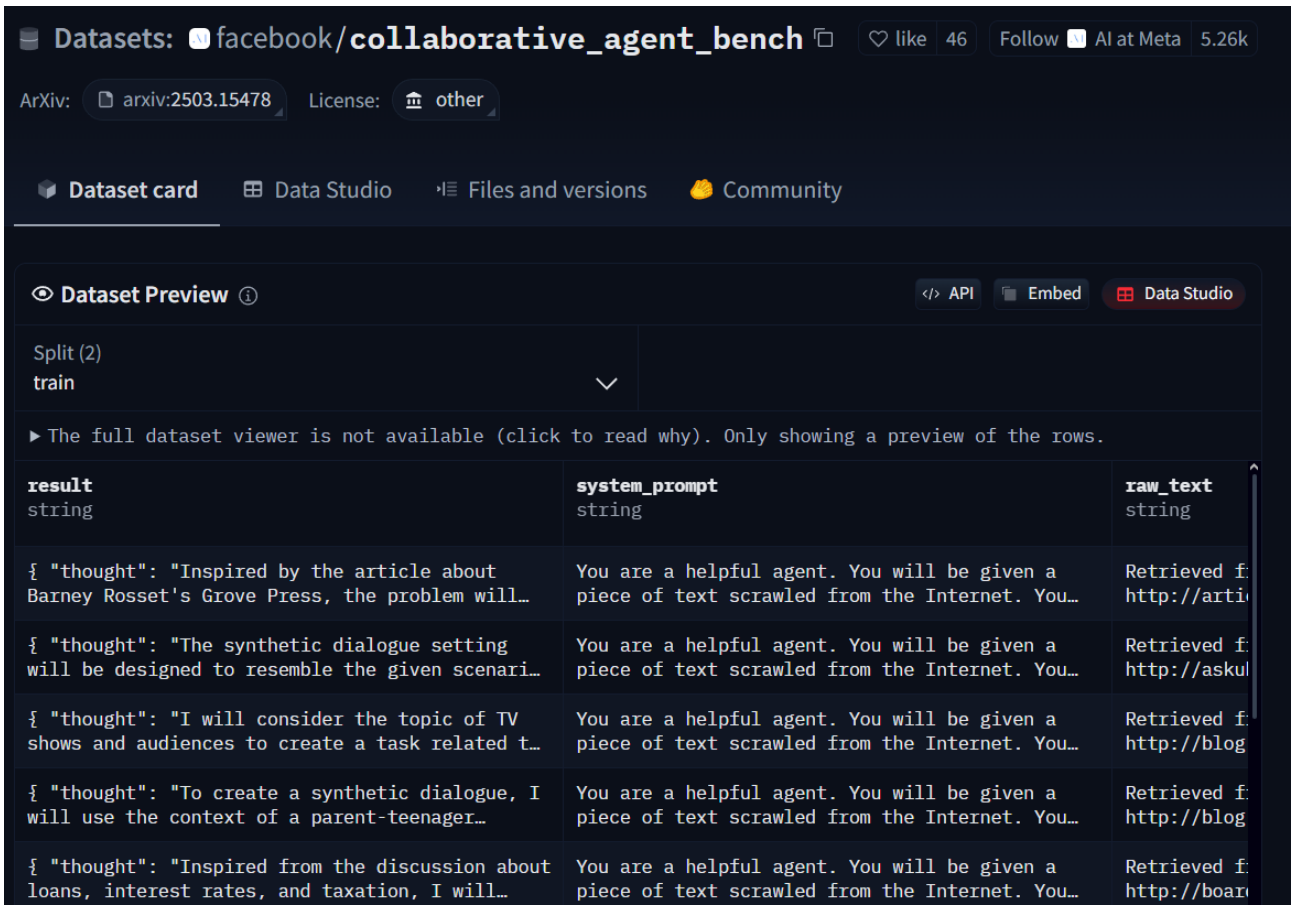


Рисунок 4 — Возможность выбора набора данных для исследования

Возможность выбора методов исследования представлена на рисунке 5.

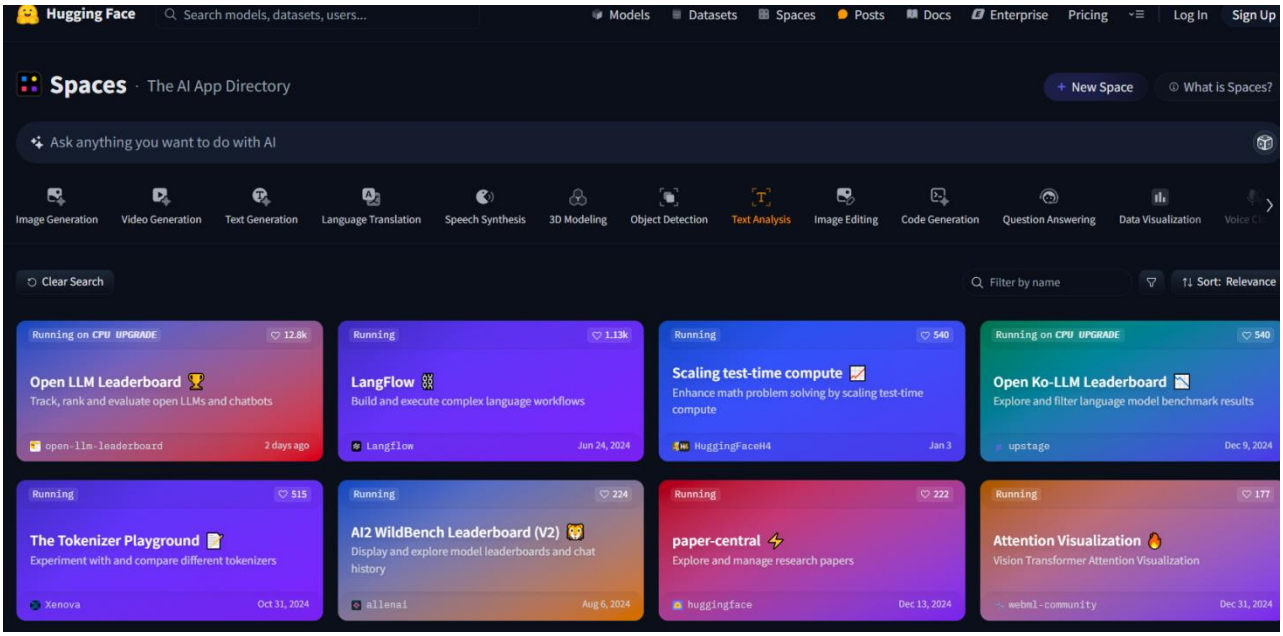


Рисунок 5 — Возможность выбора методов исследования

Возможность выбора языка, библиотеки, задачи исследования представлена на рисунке 6.

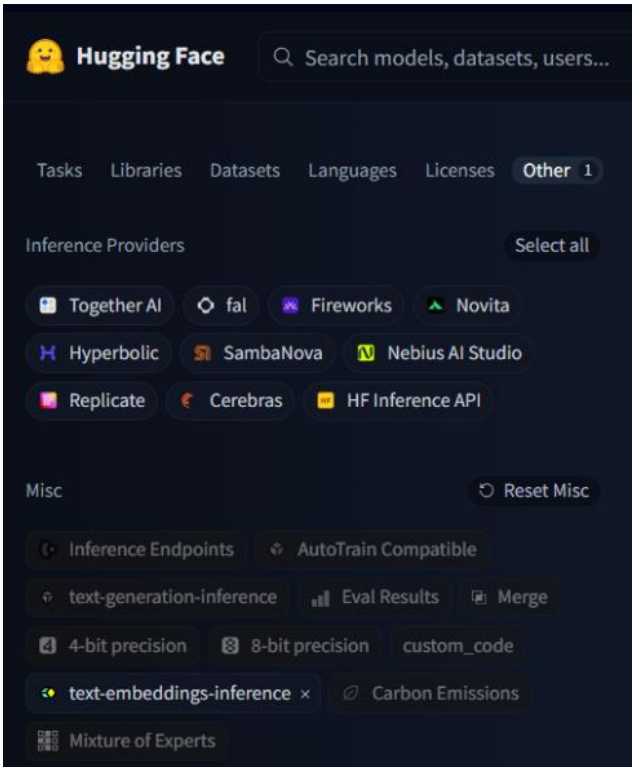


Рисунок 6 — Возможность выбора языка, библиотеки, задач исследования

На arXiv можно выбирать тексты для чтения. На рисунке 7 представлена статья 2025 года.



Рисунок 7 — Выбор статьи для чтения на arXiv

1.3 Время выполнения запросов

Снижение времени выполнения сложных запросов к базам данных становится актуальной проблемой при работе с таблицами и коллекциями с большим количеством записей [12-16,18,19]. В процессе работы информационных систем нагрузка на базы данных возрастает. Производительность серверов при многопользовательском режиме не соответствует ожиданиям пользователей. Эта проблема встречается во всех информационных системах, которые работают с большими данными. При решении проблемы компании используют такие средства, как вертикальное и горизонтальное масштабирование или оптимизацию запросов. Вертикальное и горизонтальное масштабирование значительно дороже по сравнению с оптимизацией запросов к базам данных. Оптимизация запросов при многопользовательском режиме предусматривает фрагментацию, анализ и отслеживание во времени всех запросов [20]. Таким образом, исправление ситуации долгого ожидания ответа при сложных запросах к таблицам и коллекциям с большим количеством записей требует больших материальных и временных затрат. В своей работе мы выбрали цель снизить время выполнения таких запросов на этапе обращения к базе данных;

1.4 Поиск по пространству признаков литературного источника

Механизм поиска эволюционирует от поиска по ключевым словам до современных поисковых систем с использованием контекста. Задача поискового механизма нового поколения состоит в понимании смысла запроса. Дополнение к традиционному поиску таких инструментов, как GPT-технологии расширяет его возможности. В зависимости от цели поиска должен использоваться и соответствующий инструмент поисковика. Для изучения первоисточников, исследований, продвижения науки и разработки новых методологий необходим традиционный поиск с верифицированной

информацией. Для получения краткого ответа по интересующему вопросу поможет GPT-технология, которая сгенерирует ответ из нескольких литературных источников. Обученный алгоритм векторного поиска ищет ответ на запрос в содержании сразу всей книги. «Векторизация текста полностью переведет поиск в следующее поколение (в векторизованном тексте входные данные преобразованы в векторы, понятные моделям машинного обучения, то есть с таким текстом может работать искусственный интеллект» [20].

Векторизация текста и создание векторного пространства признаков в специализированных на определенной области знаний поисковых системах становится актуальной задачей в настоящее время.

Вывод:

Необходимо учитывать особенности архитектуры будущей автоматизированной системы, используя подход, основанный на вариантах ее использования. Интеграция с другими информационными системами и платформами, такими, как Scholar.ru, Core, arXiv и ведущими библиотеками eLibrary.ru, КиберЛенинка, бесплатной поисковой системой Google Scholar позволит расширить доступ к литературным источникам. Необходимо повысить скорость выполнения запросов на чтение. Необходимо предоставить пользователю новый вид поиска, основанный на векторном пространстве признаков литературного источника.

2 ИССЛЕДОВАНИЕ И РАЗРАБОТКА АВТОМАТИЗИРОВАННОЙ СИСТЕМЫ НАУЧНО-ТЕХНИЧЕСКОЙ БИБЛИОТЕКИ

В задачи исследования на этом этапе входило:

1) определить функциональные и нефункциональные требования для автоматизированной научно-технической библиотеки и в соответствии с ними разработать архитектуру системы;

2) изучить возможности интеграции с информационной платформой arXiv;

3) построить информационные модели и логические схемы сложного SQL-запроса с применением B-tree индексов;

4) построить векторное пространство признаков для каждого литературного источника в созданной базе научных статей с помощью модели BERT;

5) обучить модель BERT рассчитывать косинусные расстояния между эмбеддингами запросов пользователя и эмбеддингами имеющихся в базе статей.

2.1 Этапы проектирования

Функциональные требования к системе

Функциональными требованиями к системе выбраны:

1) возможность регистрации и авторизации;

2) возможность поиска необходимого ресурса за время, не превышающее времени первых трех фаз зрительного анализатора человека;

3) классификация по типу ресурса (научная статья, научный том, учебник, методическое пособие, презентация, видеоматериалы);

4) разделение по типу источника (по области познания);

5) поиск по автору, названию, издательству и году издательства и предоставление ресурса для чтения;

6) поиск по DOI и предоставление ресурса для чтения, скачивания;

7) поиск по пространству признаков (смысловой поиск).

Нефункциональные требования

Нефункциональные требования к информационной системе «Электронная научно-техническая библиотека»:

1) надежность (точность, средняя наработка на отказ и т.д.); система должна выполнять свои требования не менее года до следующего техобслуживания;

2) требования к пользовательскому интерфейсу; наличие полей для расширенного поиска;

3) требования по безопасности; обеспечение безопасности личных данных читателей;

4) легкость установки, использования и обучения персонала; возможность установки системы на имеющиеся технические средства; обучение персонала в течение месяца;

5) возможность добавления новых модулей (например, новый класс источника, аудиозапись учебного материала).

Исходя из функциональных требований к АНТБ была выбрана клиент-серверная архитектура.

Спецификации

Выявляем актеров, прецеденты и их приоритеты.

Зарегистрироваться

Предусловие:

Перед началом этого прецедента пользователь/администратор должен зайти на сайт АСУ «Научная онлайн библиотека»

Главный поток:

Прецедент «Зарегистрироваться» начинается, когда пользователь/ администратор нажимает на кнопку регистрации на сайте. Поочередно выполняются подпотоки S-1 и S-2.

Подпотоки:

S-1: ввод данных для регистрации. Система отображает форму для ввода логина, почты и пароля, а также личных данных. После происходит отправка формы (E-1, E-2, E-3, E-4) и сохранение данных в БД. Возврат к началу прецедента.

S-2: подтверждение аккаунта. Система отправляет на указанную пользователем почту сообщение с ссылкой. Пользователь переходит на ссылку, тем самым подтверждая свой аккаунт (E-6). Возврат к началу прецедента.

Постусловие:

В БД появляется запись о новом пользователе.

Альтернативные потоки:

E-1: введен некорректный пароль. Пользователь может повторить ввод пароля или прекратить прецедент.

E-2: введена некорректная почта (логин). Пользователь может повторить ввод почты или прекратить прецедент.

E-3: введено некорректное имя. Пользователь может повторить ввод имени или прекратить прецедент.

E-4: ссылка оказалась недействительной. Пользователь может подождать и перейти по ссылке позже или прекратить прецедент.

Авторизоваться

Предусловие:

Перед началом этого прецедента должен быть выполнен прецедент «Зарегистрироваться».

Главный поток:

Прецедент «Авторизоваться» начинается, когда пользователь нажимает на кнопку авторизации, ввод данных для авторизации. Подпотоки: S-1: ввод данных для авторизации. Система отображает форму для ввода логина(почты) и пароля. После происходит отправка формы (E-1, E-2). Возврат к началу прецедента.

Постусловие:

Создание cookie с логином (почтой) и паролем пользователя.

Альтернативные потоки:

Е-1: введен неверный логин (почта). Пользователь может повторить ввод логина (почты) или прекратить прецедент.

Е-2: введен неверный пароль. Пользователь может повторить ввод пароля или прекратить прецедент.

Редактировать профиль:

Предусловие:

Перед началом этого прецедента пользователь должен пройти авторизацию.

Главный поток:

Прецедент «Редактировать профиль» начинается, когда пользователь заходит в свой профиль и нажимает кнопку «Редактировать». Выполняется подпоток S-1: ввод данных по изменению профиля

Происходит слияние двух главных потоков:

Пользователь должен пройти повторно авторизацию

После происходит отправка формы (Е-1, Е-2) и сохранение данных в БД. Возврат к началу прецедента.

Постусловие:

В БД появляется запись пользователя с изменёнными полями.

Альтернативные потоки:

Е-1: введён некорректный пароль. Пользователь может повторить

ввод пароля или прекратить прецедент.

Е-2: введён некорректный логин (почта). Пользователь может повторить ввод логина (почты) или прекратить прецедент.

Поиск ресурса (на примере «Поиск по автору, названию и году»):

Предусловие:

Перед началом этого прецедента пользователь должен пройти авторизацию.

Главный поток:

Прецедент «Поиск ресурса» начинается, когда пользователь нажимает кнопку заходит в свой профиль и нажимает кнопку «Поиск ресурсов», открывается поле с формой (поля: «Тип ресурса» (обязательное поле), «Автор» (обязательное поле), «Название ресурса» (обязательное поле), «Год издания», «ISBN», «ISSN», «DOI», «УДК») заполняет поля в открывшемся окне и нажимает кнопку «Поиск». Извлекаются данные запроса, происходит поиск в БД, возвращается результат поиска search.html (первая страница ресурса). Открывается окно выбора: с кнопками «читать» или «скачать». Пользователь нажимает одну из двух кнопок. Выполняется подпоток S-1 (если выбрана кнопка «Чтение»): поиск ресурса по введённым данным в БД системы:

S-1 - ресурс найден и пользователю возвращается файл для чтения ресурса, подкачиваются страницы ресурса в процессе чтения

S-2 (если выбрана кнопка «Скачать»): в системе создаётся ссылка_путь до файла в файловой системе, пользователю предлагается окно с кнопкой «Скачать», пользователь нажимает на эту кнопку, происходит скачивание в оперативную память системы, а затем скачивание на машину пользователя

После происходит отправка формы (E-1, E-2, E-3, E-4, E-5). Возврат к началу прецедента.

Альтернативные потоки:

E-1: не выбран тип ресурса. Пользователь может выбрать тип ресурса или прекратить прецедент.

E-2: не выбран автор ресурса. Пользователь может выбрать автора ресурса или прекратить прецедент.

E-3: пользователь не нажал повторно кнопку «Скачать», система отсчитывает установленный интервал времени и возвращается на начало прецедента или пользователь может прекратить прецедент.

E-4: сбой в системе электропитания, система сохранила последние введенные данные и продолжила прецедент или пользователь может прекратить прецедент.

Чтение, скачивание ресурса:

Предусловие:

Перед началом этого прецедента пользователь должен пройти прецедент «Поиск ресурса»

Главный поток:

Пользователь получает ссылку на чтение/скачивание ресурса нажимает кнопку «читать»/«скачать».

Скачивается/(предоставляется файл для чтения) ресурс по указанному пользователем пути. Возврат к началу прецедента

После происходит отправка формы (E-1, E-2).

Альтернативные потоки:

E-1: Файл не найден. Пользователь изменил ссылку, может повторить

Ввод ссылки или прекратить прецедент.

E-2: Файл не найден. Администратор одновременно с запросом пользователя удалил ресурс. Пользователь может прекратить прецедент.

Добавить ресурс (для администратора):

Предусловие:

Перед началом этого прецедента администратор должен пройти авторизацию.

Главный поток:

Прецедент **«**Добавить ресурс» начинается, когда администратор заходит в свой профиль и нажимает кнопку «Добавить ресурс», заполняет поля в открывшемся окне и нажимает кнопку «Поиск». Выполняется подпоток S-1: поиск ресурса по введённым данным

Подпотоки:

S-1: ввод данных для заполнения полей в открывшемся окне. Система отображает форму для ввода типа ресурса, название ресурса, автора ресурса, DOI, ISBN, ISSN, SICI.

После происходит отправка формы (E-1, E-2, E-3, E-4, E-5) вывод ссылки для скачивания. Возврат к началу прецедента.

Альтернативные потоки:

E-1: введено некорректное название ресурса. Администратор может повторить ввод названия ресурса или прекратить прецедент.

E-2: не выбран тип ресурса. Администратор может выбрать тип ресурса или прекратить прецедент.

E-3: не выбран автор ресурса. Администратор может выбрать автора ресурса или прекратить прецедент.

E-4: не выбран ISBN. Администратор может внести ISBN или прекратить прецедент.

E-5: не выбран DOI (УДК). Администратор может повторить DOI (УДК) или прекратить прецедент.

Удалить ресурс (для администратора):

Предусловие:

Перед началом этого прецедента администратор должен пройти авторизацию.

Главный поток:

Прецедент **«**Удалить ресурс» начинается, когда администратор заходит в свой профиль и нажимает кнопку «Удалить ресурс», заполняет поле id- ресурса открывшемся окне и нажимает кнопку «Поиск». Выполняется поиск ресурса по id и удаление ресурса нажатием кнопки «удалить».

Альтернативные потоки:

E-1: система возвращает null, если ресурс уже удален.

Поиск по пространству признаков:

Предусловие:

Пользователь выбирает поле для семантического поиска.

Главный поток:

Пользователь заводит произвольный контекст для поиска и нажимает кнопку «Поиск». Выполняется поиск 5 релевантных статей и вывод на экран пользователя.

Альтернативные потоки:

E-1: система возвращает сообщение, что по данной тематике не найдено ресурсов, если запрос пользователя не соответствует ни одной тематике статей, имеющихся в базе данных.

Расширенный поиск:

Предусловие:

Пользователь выбирает поле arXiv для расширенного поиска.

Главный поток:

Пользователь попадает на страницу arXiv и заводит короткий текст запроса. Нажимает кнопку «Поиск». Выполняется поиск отсортированных по дате 50 и более статей.

Альтернативные потоки:

E1: система возвращает сообщение, что не найдено сообщений по данному запросу, если пользователь заводит слишком длинный запрос.

E2: система возвращает сообщение, что не найдено сообщений по данному запросу, если запрос пользователя не соответствует ни одной тематике статей, имеющихся в базе данных.

Диаграмма прецедентов

Для отображения основных функций системы и ролей, которые будут с ней взаимодействовать составляем диаграмму прецедентов. На рисунке 8 представлена диаграмма прецедентов.

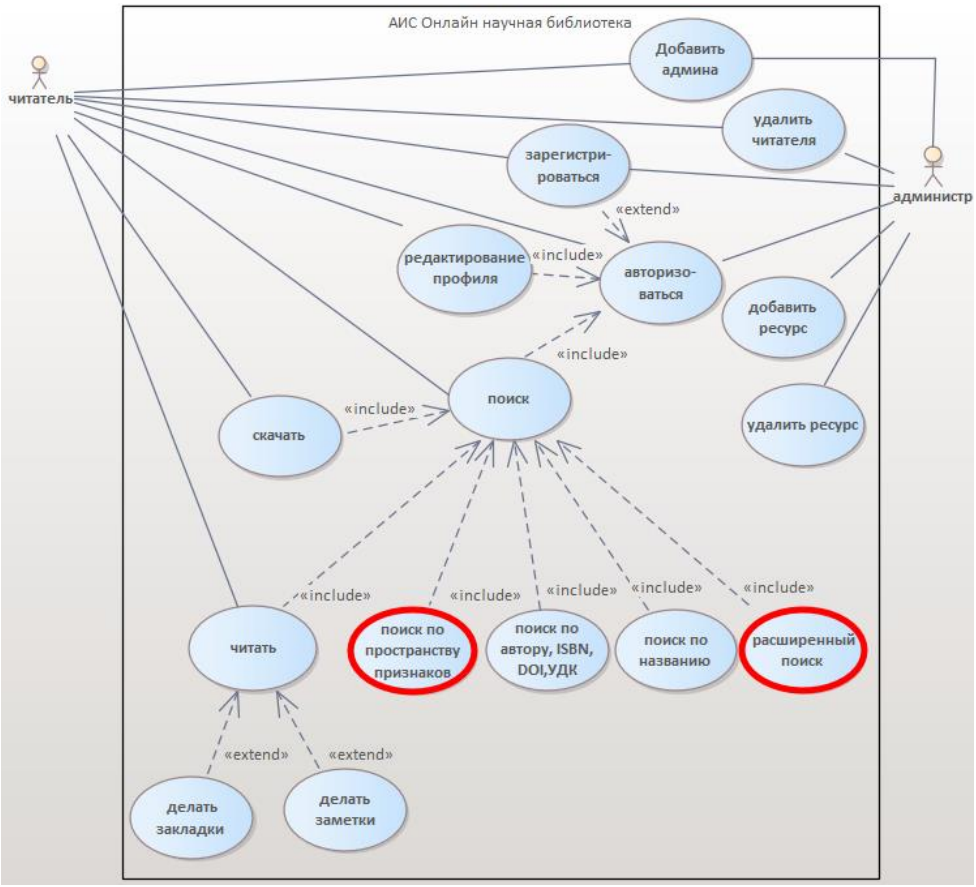


Рисунок 8 — Диаграмма прецедентов

Описание архитектуры

Так как поиск источника предполагает разные типы вводимых данных и различные варианты показателей поиска, то предлагается **клиент-серверная структура** по принципу «**толстый клиент».**

Клиенту предлагается форма для заполнения интересующих его полей поиска источника. Заполненная форма источника является формирователем запроса для поиска. Сформированная страница на стороне пользователя направляется на сервер. На сервере определяется тип запрашиваемого источника. Также на сервере данные из запроса преобразуются в данные, доступные для обработки в базе данных. Такой преобразованный запрос отправляется в базу данных. Создается унифицированный интерфейс вместо набора интерфейсов. Таким образом, мы выбрали **структурный паттерн Фасад** для организации процесса получения и обработки данных от

пользователя. На рисунке 9 представлен процесс создания и обработки запроса от пользователя с использованием структурного паттерна Фасад.

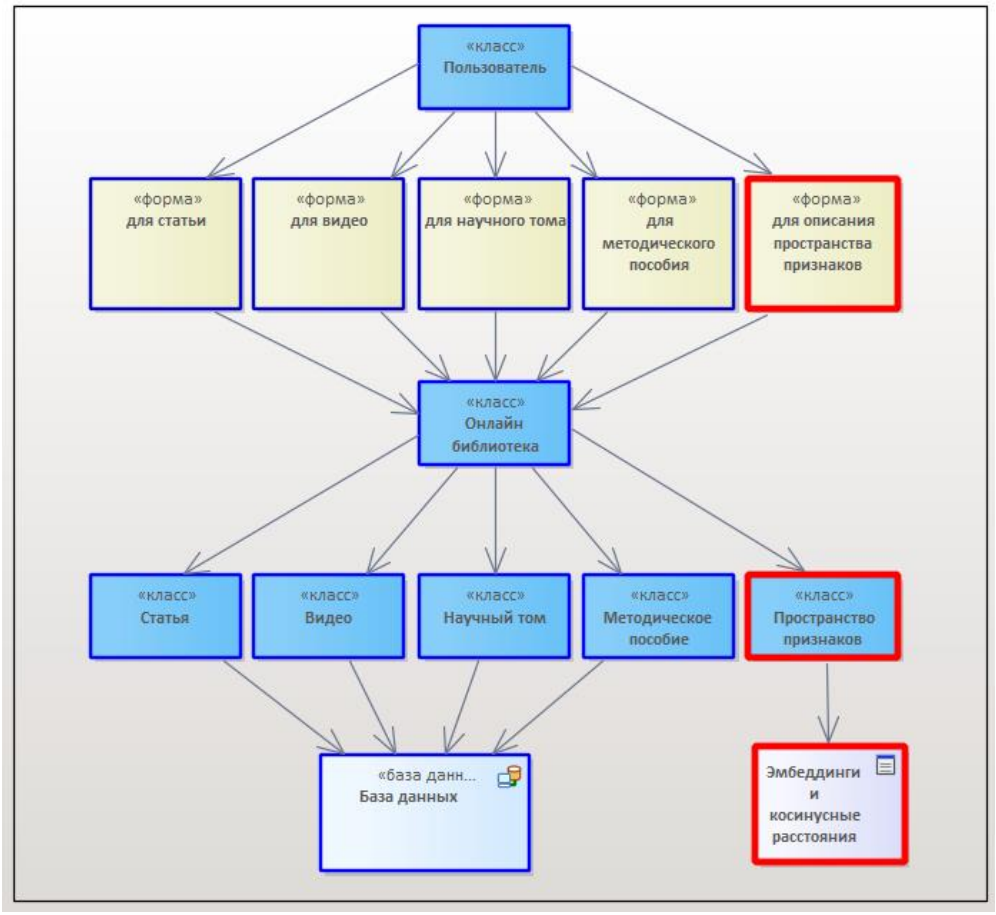


Рисунок 9 — Процесс создания и обработки запроса от пользователя с использованием структурного паттерна Фасад

Разработка системы выполнена в соответствии с описанием ее функциональности, использовались паттерны бизнес-логики («Domain Model»), работы с базой данных («Data Mapper»), структурный паттерн («Фасад»).

Паттерн Domain Model (модель области определения) предназначен для отображения данных и области их применения. В предметной области выявляются значащие сущности, из которых создаются объекты. Объекты находятся во взаимосвязи между собой. Паттерн Domain Model предполагает подробное изучение сущностей и на основе этого создание объектов. Так создание объекта класса Article реализуется через класс Resource. Также паттерн Domain Model предполагает использование наследования классов. Таким образом, паттерн Domain Model помогает реализовывать принципы ООП.

Для работы с базой данных применялся паттерн «Data Mapper».

Диаграмма предметной области

На рисунке 10 представлена диаграмма предметной области информационной системы «Электронная научно-техническая библиотека».

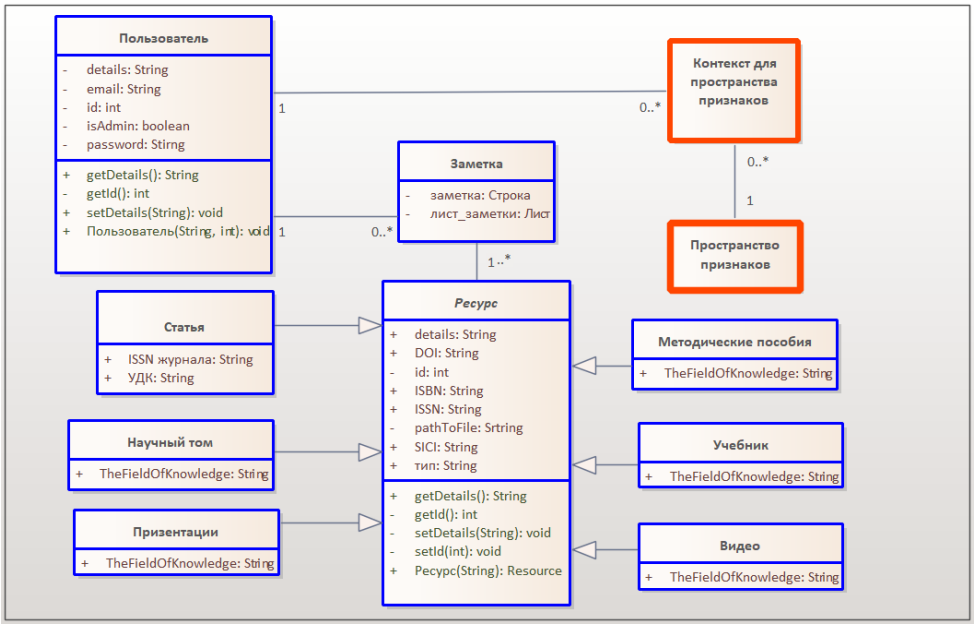


Рисунок 10 — Предметная область информационной системы «Электронная научно-техническая библиотека»

Диаграмма классов анализа системы

Разрабатываем диаграмму классов анализа системы. На рисунке 11 представлена диаграмма классов анализа АНТБ.

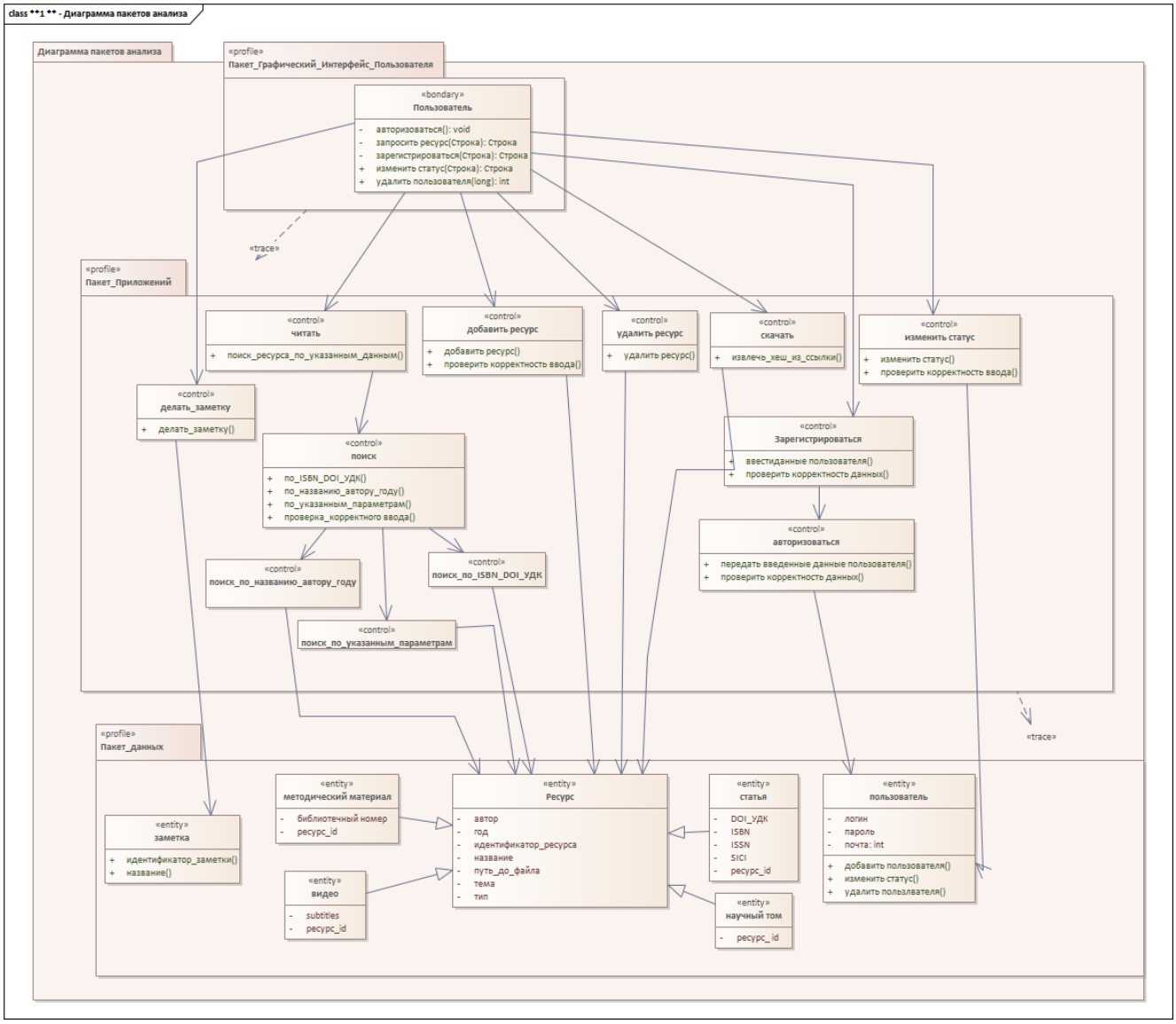


Рисунок 11 — Диаграмма классов анализа АНТБ

Модель трассировки классов анализа в классы проектирования

На рисунке 12 представлена диаграмма трассировки классов анализа в классы проектирования.

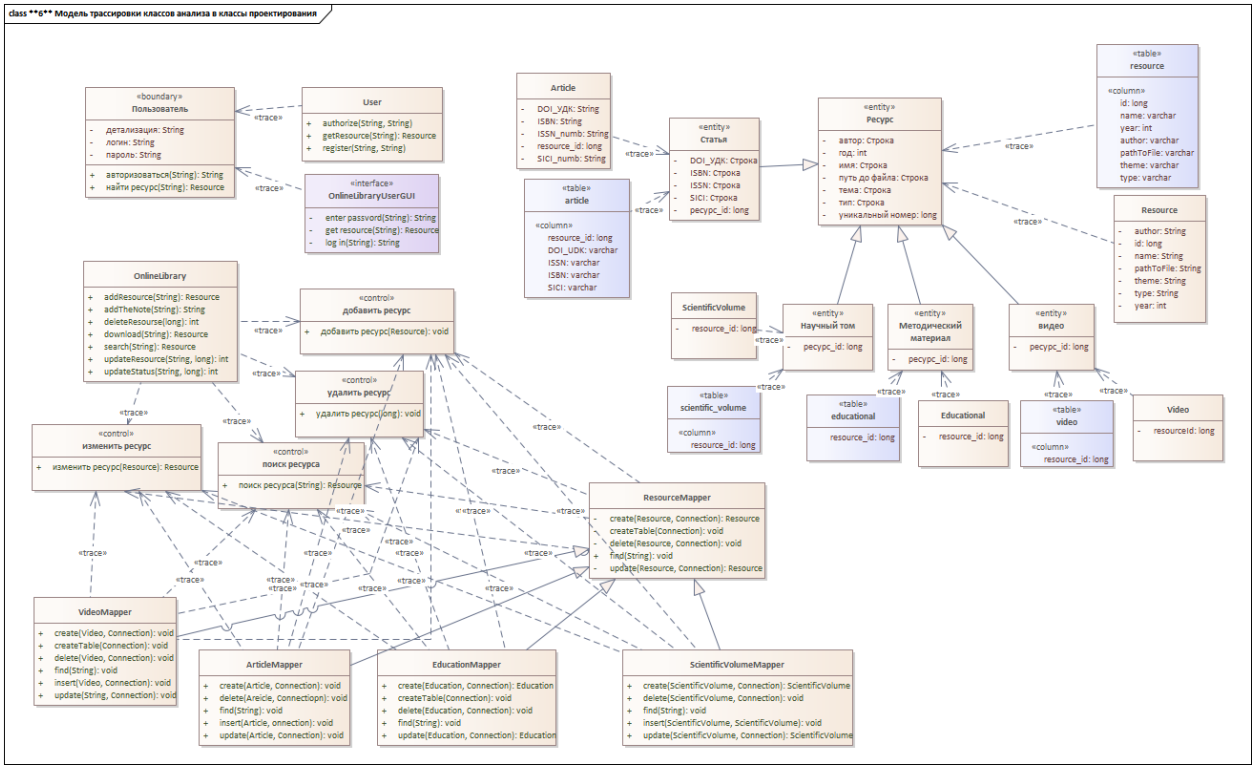


Рисунок 12 — Диаграмма трассировки классов анализа в классы проектирования

Диаграмма уровней подсистем

Построить диаграмму уровней подсистем. На рисунке 13 представлена диаграмма уровней подсистем АНТБ.

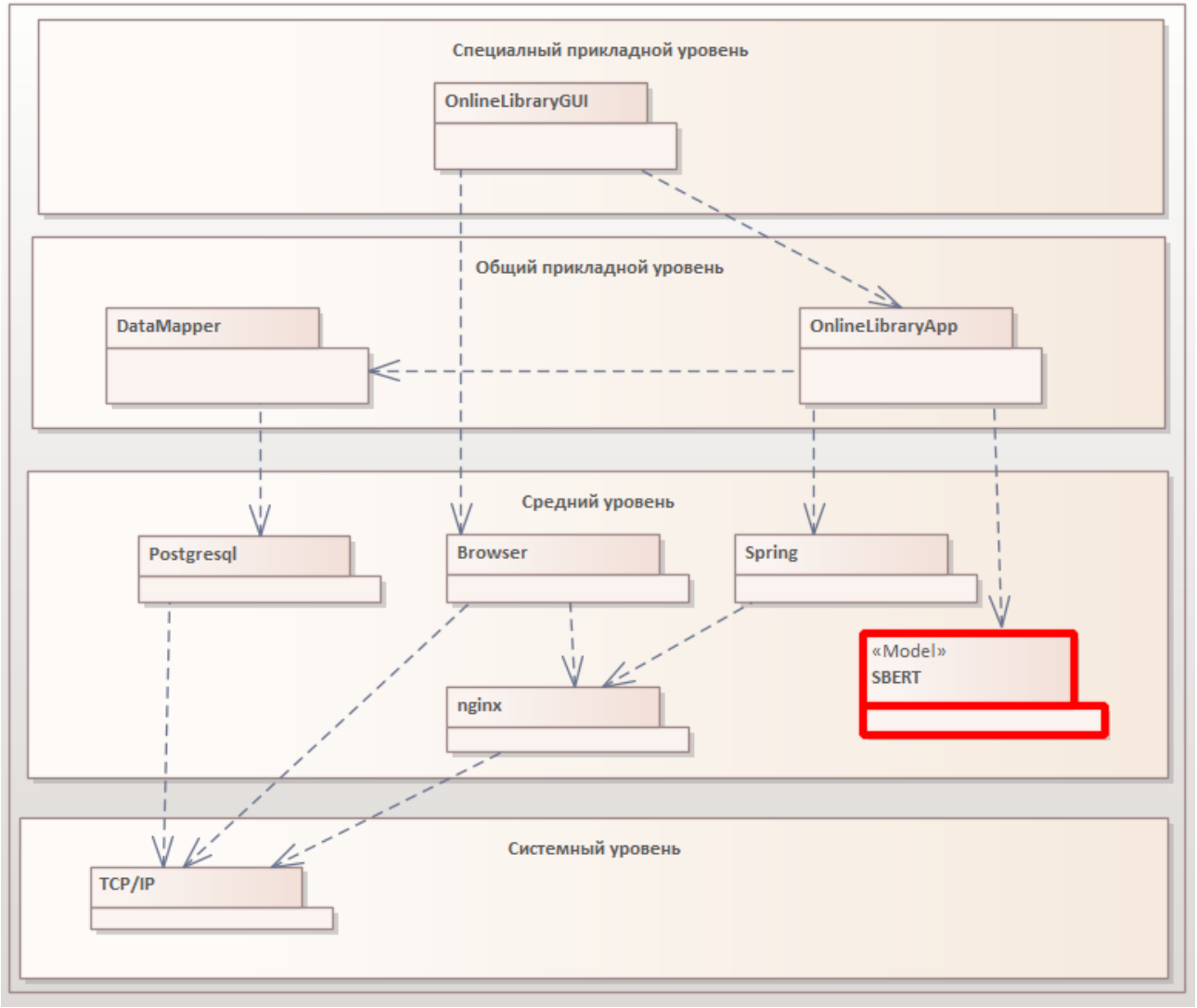


Рисунок 13 — Диаграмма уровней подсистем АНТБ

Диаграмма классов проектирования в подсистемах

Распределение классов проектирования по подсистемам представлено на рисунке 14.

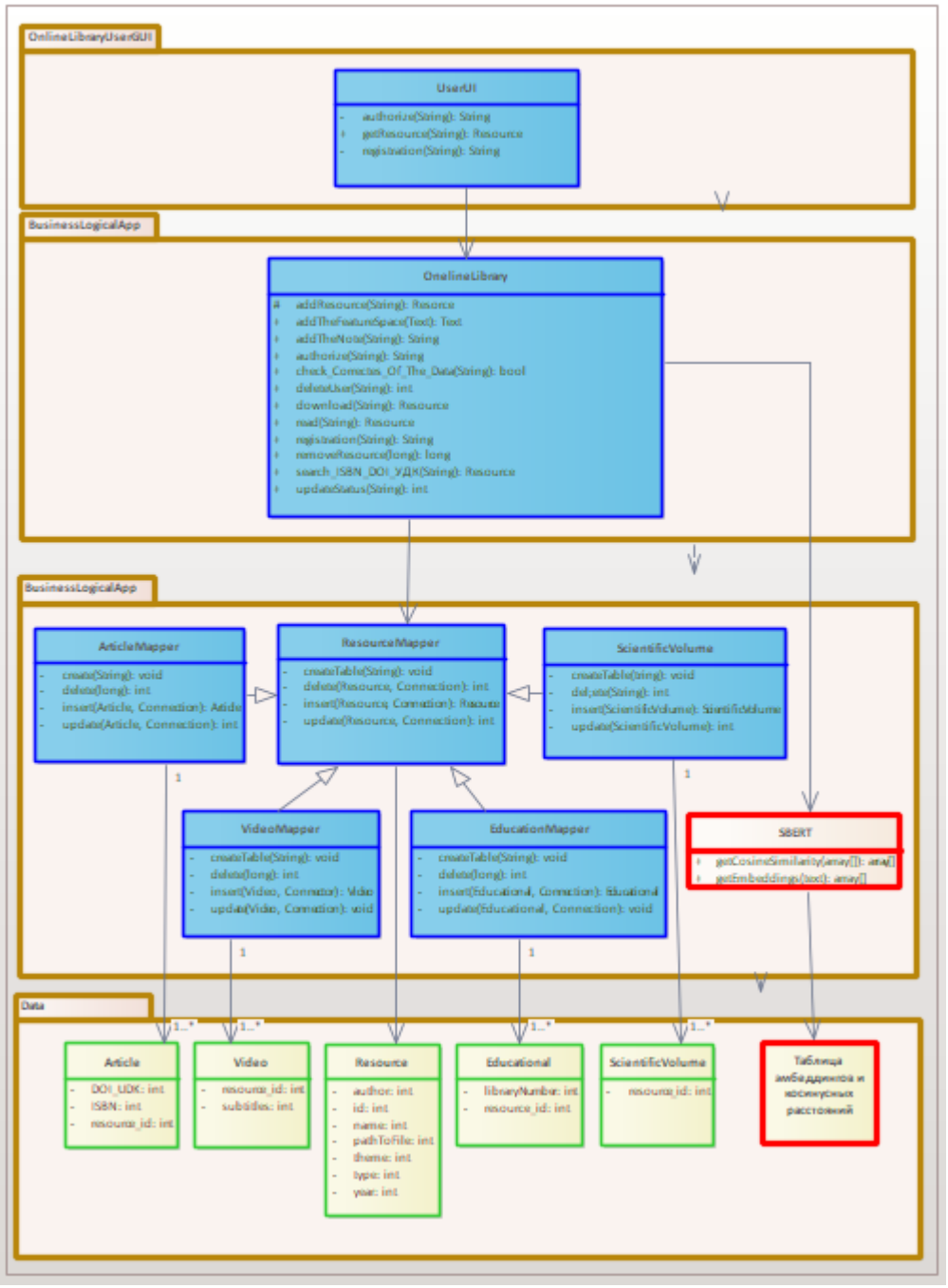


Рисунок 14 — Диаграмма распределения классов проектирования по подсистемам

Определить набор компонентов

Построенная модель трассировки подсистем в компоненты представлена на рисунке 15).

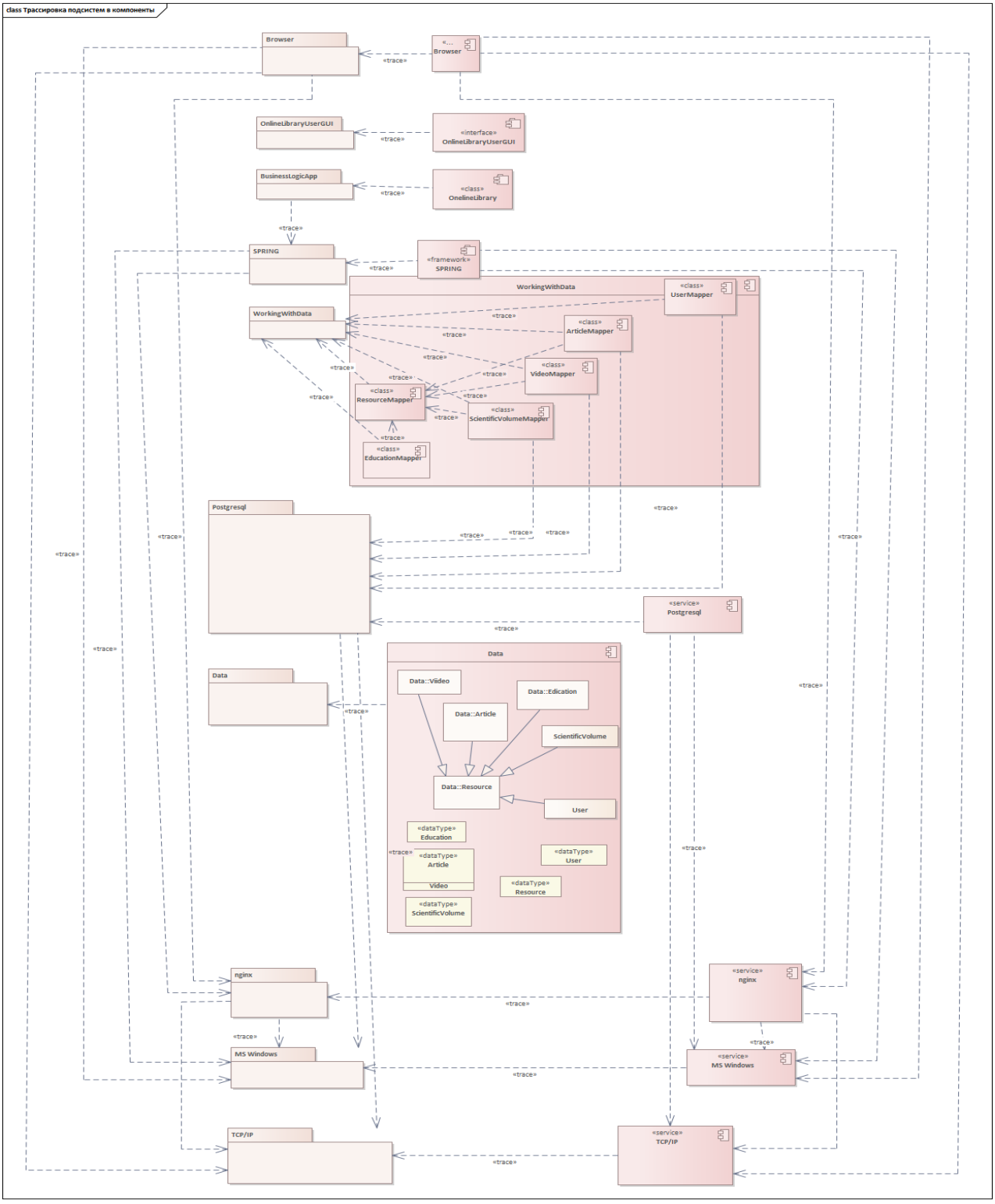


Рисунок 15 — Модель трассировки подсистем в компоненты

Диаграмма развертывания

Для реализации программного продукта был выбран язык Java – как мультиплатформенный и хорошо знакомый команде разработки. В качестве

СУБД была выбрана PostgreSQL. Диаграммы уровней подсистем и развертывания представлены на рисунках 16 и 17.

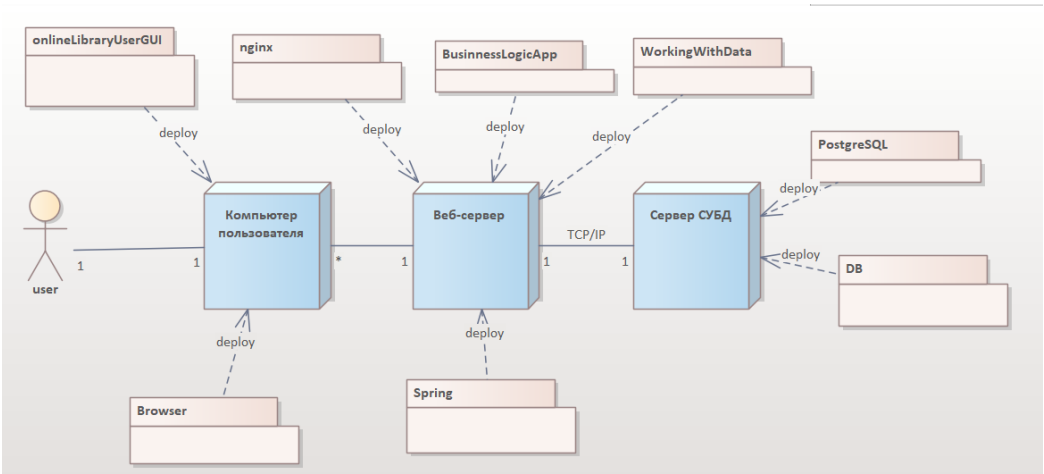


Рисунок 16 — Диаграмма уровней подсистем

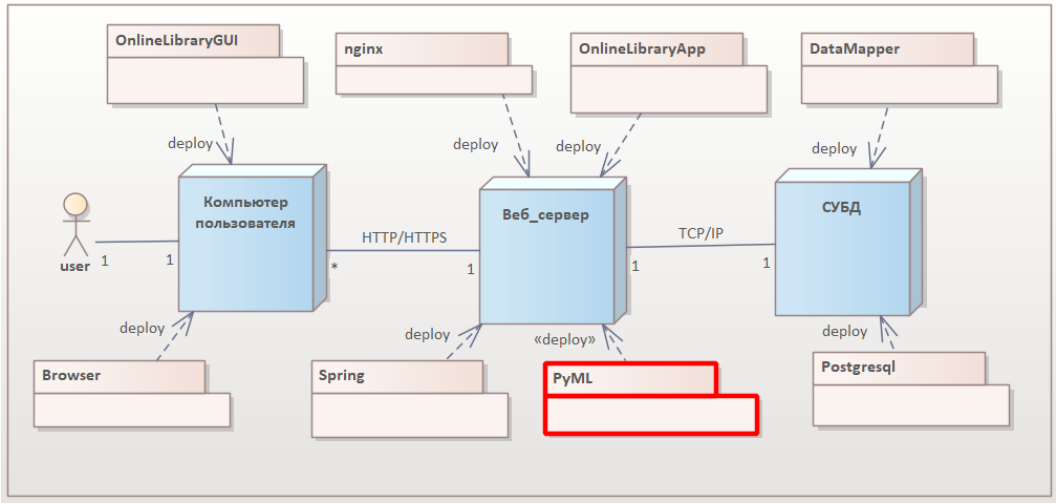


Рисунок 17 — Диаграмма уровней подсистем

2.2 Исследование возможностей интеграции с бесплатной информационной системой arXiv

Традиционная форма поисковой строки с полями для метаданных искомого литературного источника соответствует ограничениям возможностей поиска. Например, необходимо знать все метаданные литературного источника. Также ошибочное введение символа или слова снижает качество поиска. Выдается очень ограниченное количество источников. Частая недоступность

информации в открытом доступе создавала проблемную ситуацию для исследователей, преподавателей и учащихся. Появились движение за открытый доступ. Появились онлайн библиотеки с открытым доступом, некоммерческие журналы. В результате произошло ускорение обмена научными знаниями и, следовательно, ускорение развития науки и техники. Одной из таких инициатив было создание информационной платформы arXiv, которая объединяет 12 лабораторий, каждая из которых развивается в своем научном направлении.

Используем возможность выбора моделей, набора данных, методов исследования на Hugging Face Hub, лаборатории arXiv. В разделе Computer Science выбираем область знания Machine Learning или в поисковой строке заводим ключевые слова. За один клик система arXiv возвращает 6183 актуальных статьи в специализированной теме на рисунке 18 и 12990 статей в более широкой специализации на рисунке 19.

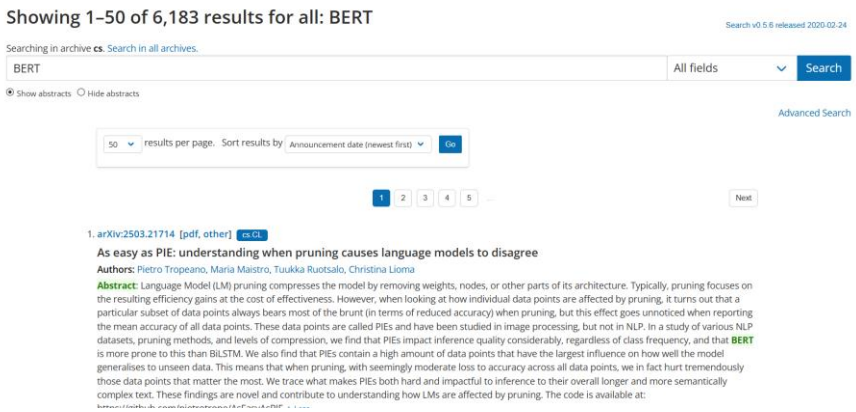


Рисунок 18 — Система arXiv возвращает 6183 актуальных статьи в специализированной теме

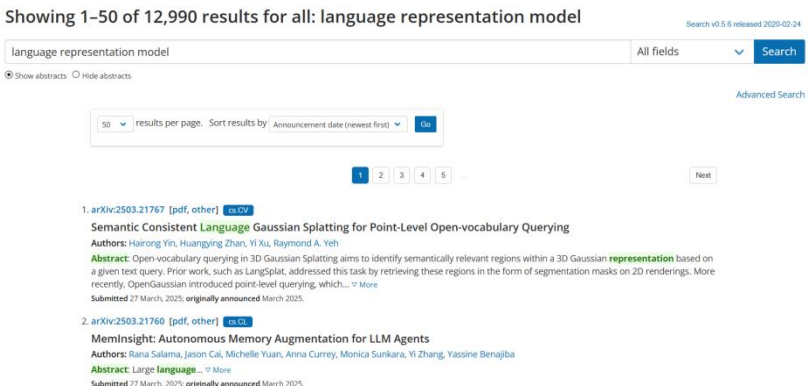


Рисунок 19 — Система arXiv возвращает 12990 актуальных статьи в более широкой специализации

Достоинства интеграции с arXiv:

1) бесплатный доступ ко всем статьям на информационной платформе;

2) быстрый обмен научной информацией;

3) возможность за один клик получить большое количество аннотаций с метаданными статей;

4) существует возможность сортировки поиска по дате публикации. Собрав список статей для машинного обучения на модели BERT, получаем возможность подготовить датасет;

5) на выход подаются аннотация статьи со всеми ее метаданными. Это очень удобно для выполнения ссылок;

6) возможность выкладывать свои научные достижения и разработки.

Недостатки интеграции с arXiv:

1) поиск проводится по совпадению ключевых слов. Так, например, SQL имеет неоднозначную расшифровку Structured Query Language и SQLong - Semantic Query Language for Long Documents., новая и эффективная платформа для расширения данных, однако обе статьи отнесены в тематику Structured Query Language; RAM - Random Access Memory, оперативная память компьютера и RAM - Reliability, Availability, Maintainability — надёжность, доступность, ремонтопригодность. Такая семантическая неоднозначность часто встречающихся аббревиатур значительно снижает поиск;

2) такие часто встречающиеся аббревиатуры, как ПО, AI, HTTP и другие на стыке разных областей знания, также снижают точность поиска.

2.3 Исследование возможностей повысить скорость выполнения запросов на чтение

Время ожидания результата запроса при поиске литературных источников становится краеугольным камнем в борьбе за количество пользователей веб-приложением. Поэтому, исследование возможности

повысить скорость выполнения поиска логично для ожидания материальной пользы.

Модель наших запросов с использованием B-tree индексов предполагает выполнение определённых условий оптимизации. Исследование проводилось на запросах к PostgreSQL и MongoDB. Особенности архитектуры СУБД PostgreSQL и СУБД MongoDB определили разный подход в изучении оптимизации сложных запросов к базам данных с большим количеством записей. Репликации и шардирование позволяют СУБД MongoDB использовать дополнительную оперативную память в многопользовательском режиме работы. Подсистема хранения данных WiredTiger обеспечивает оптимальное использование оперативной памяти и сокращает количество обращений к файловой системе. Мы исключили шардирование для чистоты исследования времени работы WiredTiger, времени выполнения одиночных сложных запросов и времени выполнения этих же запросов в серии запросов в MongoDB.

Особенности структуры данных, B-tree в PostgreSQL и B+tree в MongoDB, позволили провести сравнительное исследование запросов с применением этих индексов в сложных запросах в обеих СУБД. В СУБД PostgreSQL используется структура данных B-tree. В СУБД PostgreSQL файлы данных разбиваются на сегменты размером сегмента 1 ГБ. Данные хранятся на страницах фиксированного размера 8 КБ. Для каждой строки в СУБД PostgreSQL создается внутренний идентификатор. Первая цифра в идентификаторе — это номер страницы, а вторая — номер кортежа. Данные представляют строки, для доступа к ним используются ссылки. Ссылки содержат логическое смещение на странице. В отдельной области хранятся индексы и ссылки на соседние узлы в B-tree. Вместо таблиц СУБД MongoDB использует коллекции, где хранятся документы; MongoDB хранит данные в виде JSON-документов с различным уровнем глубины, кодируя их в бинарном формате BSON, у документов в коллекции может различаться набор полей и их типы данных; В СУБД MongoDB вместо соединения таблиц с оператором JOIN применяется сохранение нескольких документов в одну коллекцию, чтобы потом работать с

ней. Запросы к документам с вложенными полями считаются сложными запросами. В СУБД MongoDB используется структура данных B+tree. При создании документа в MongoDB создается идентификатор объекта. Его 12-байтовая формула ObjectId включает в себя 4-байтовую отметку времени создания, 3-байтовый машинный идентификатор, 2 обработанных байта и 3-байтовый счетчик, начинающийся со случайного значения. Индексы в MongoDB занимают много памяти и затратны при считывании.

На рисунке 20 представлены структуры данных B-tree и B+tree.

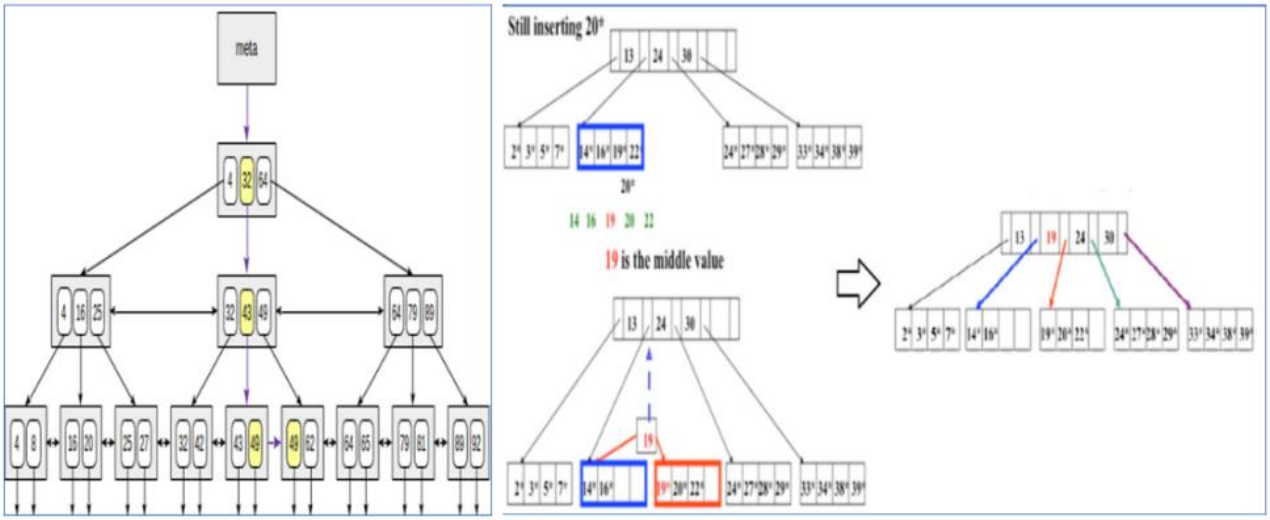


Рисунок 20 — Структуры данных B-tree и B+tree

На рисунке 20 (справа) изображен процесс сегментирования для дальнейшего шардирования и горизонтального масштабирования в MongoDB. Для оптимизации работы в оперативной памяти в MongoDB разработана подсистема хранения данных WiredTiger. WiredTiger хранит данные таблицы в памяти, используя структуру данных, называемую B+tree, ссылаясь на узлы B+tree как на страницы. Внутренние страницы содержат только ключи. Конечные страницы хранят как ключи, так и значения. Также поддерживается сжатие для всех коллекций и индексов. Сжатие минимизирует использование хранилища за счёт дополнительного ЦП. WiredTiger кэширует часть таблицы, к которой в данный момент обращается приложение для чтения или записи в основной памяти. Таким образом, сокращает число обращений к диску,

следовательно, сокращается время выполнения сложных запросов к коллекциям с большим числом записей.

Оптимизатор может выбрать план полного сканирования таблицы данных, сканирование по индексу, сканирование битовых карт, сканирование по частичному индексу, по покрывающему индексу и другие планы. От вариантов составления запросов зависит этот выбор. Количество строк и страниц таблицы, которые будут извлечены во время сканирования, будем рассматривать, как изменяемые характеристики в запросе поиска. Исследование документации позволяет сделать предположение, что на количество строк и страниц таблицы влияют такие факторы, как селективность запроса, учёт видимости NULL значений, степень упорядоченности данных, фильтрация данных и использование покрывающих (составных) индексов по нескольким колонкам таблицы. Эти факторы влияют на стоимость запроса в целом, следовательно, на выбор оптимизатором плана выполнения запроса.

2.3.1 Информационная модель сложного SQL-запроса с применением B-tree индексов, как объект теоретического исследования

Основная цель информационной модели — моделирование управляемых объектов на концептуальном уровне, независимо от каких-либо конкретных реализаций или протоколов, используемых для передачи данных.

В качестве изучаемых критериев оптимизации сложных запросов к таблицам и коллекциям с большим количеством записей были выбраны:

1) селективность;

2) скрытие NULL значений от планировщика;

3) покрытие условия запроса;

4) серийность сложных запросов.

Изучение функций метода доступа к индексу B-tree в документации PostgresPro [21,22,23,24,25,26,27] позволило нам выбрать факторы, влияющие

на стоимость доступа к таблице индексов в PostgreSQL в информационную модель, представленную на рисунке 21.

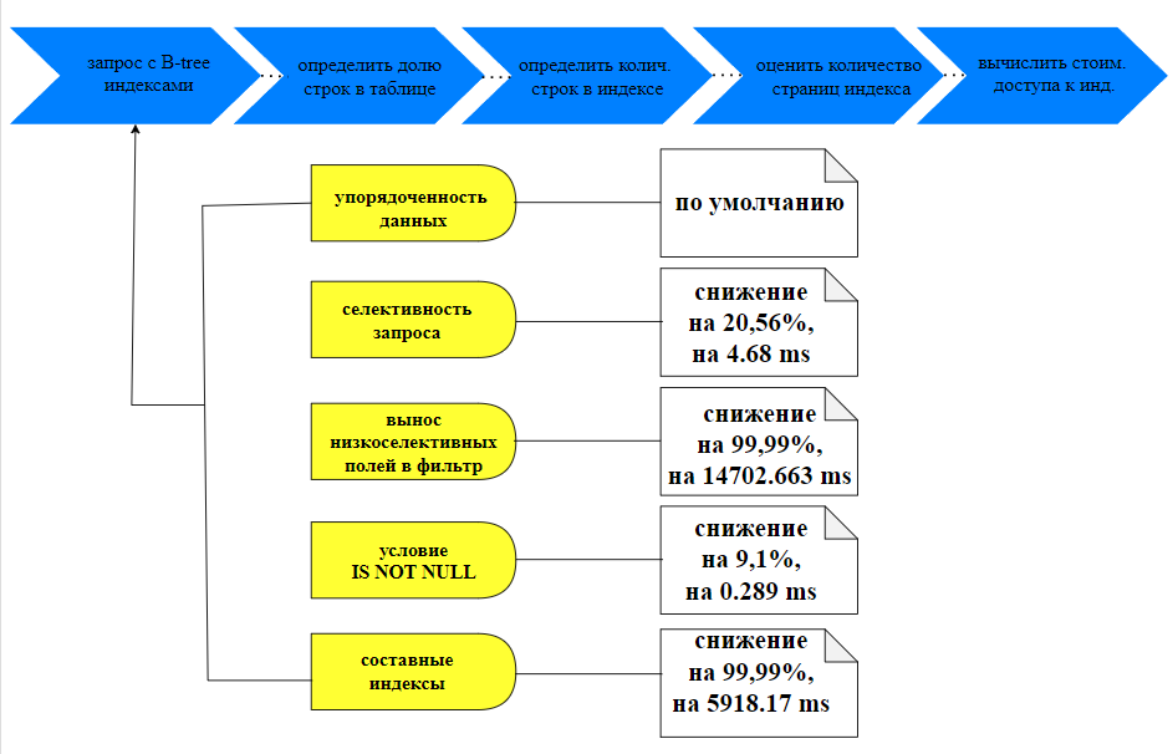


Рисунок 21 — Информационная модель метода доступа к индексу B-tree в PostgreSQL

В MongoDB мы исключили горизонтальное масштабирование для чистоты эксперимента и исключения влияния факторов распараллеливания запросов на время выполнения запроса. Поэтому в информационной модели для СУБД MongoDB будет рассматриваться только запрос на одном компьютере. Изучение функций метода доступа к индексу B+tree в документации MongoDB [28,29,30] позволило нам выбрать факторы, влияющие на стоимость доступа к таблице индексов в СУБД MongoDB в информационную модель, представленную на рисунке 22.

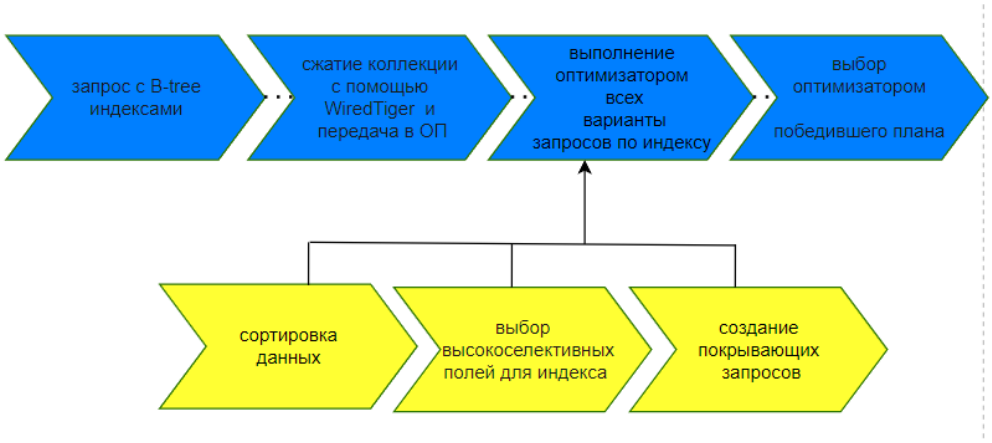


Рисунок 22 — Информационная модель доступа к индексам и к коллекции MongoDB

Рассчет времени выполнения сложного запроса с использованием индекса B-tree в PostgreSQL в информационной модели с учетом стоимости доступа к индексу производится по формуле 1.

$$T_{req_index} = t_{plan} + (t_{index_read} + t_{check_ref}) * n + [t_{inp_oup} + t_{system}], \quad (1)$$

где T_{req_index} — время чтения коллекции,

t_{plan} — время работы планировщика,

t_{index_read} — время чтения таблиц индекса,

t_{check_ref} — время чтения таблицы

t_{inp_oup} — время на операции ввода/вывода,

t_{system} — время, зависящее от пропускной способности системы,

n — число соединенных таблиц.

Рассчет времени выполнения сложного запроса с использованием индекса B+tree в MongoDB в информационной модели с учетом стоимости доступа к индексу производится по формуле 2.

$$T_{req_index} = t_{WT} + t_{plan} + (t_{index_read} + t_{check_ref}) * n + [t_{inp_oup} + t_{system}] \quad (2)$$

где T_{req_index} — время чтения коллекции,

t_{WT} — время работы Wired Tiger,

t_{plan} — время работы планировщика,

t_{index_read} — время чтения индекса,

t_{check_ref} — время чтения документа в коллекции

t_{inp_oup} — время на операции ввода/вывода,

t_{system} — время, зависящее от пропускной способности системы

2.3.2 Логические схемы выбора оптимизатором плана выполнения запроса

В результате предварительного эксперимента был проведен анализ планов выполнения запросов в обеих СУБД, построены логические схемы выбора оптимизатором плана выполнения запроса с B-tree индексами в СУБД PostgreSQL и СУБД MongoDB имеют отличия. На рисунке 23 представлена логическая схема выбора оптимизатором плана выполнения запроса с B-tree индексами в СУБД PostgreSQL.

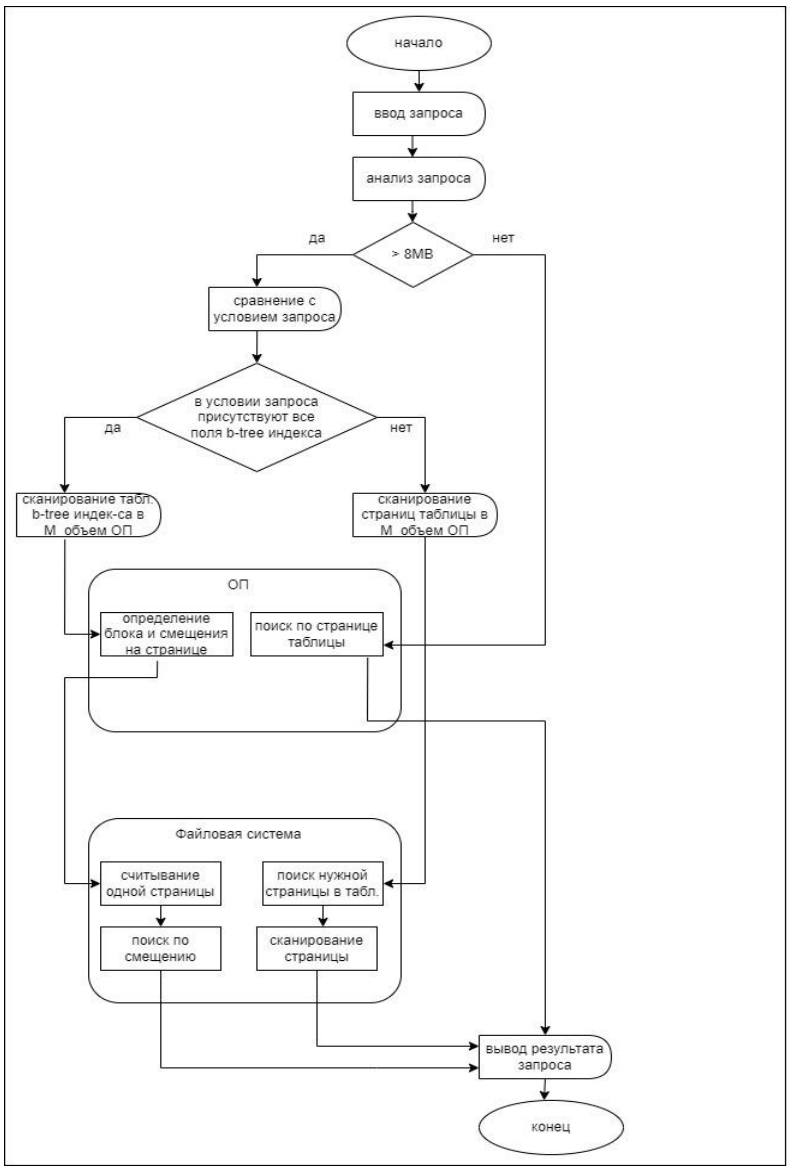


Рисунок 23 – Логическая схема выбора оптимизатором плана выполнения запроса с B-tree индексами в СУБД PostgreSQL

На рисунке 24 представлена логическая схема выбора оптимизатором плана выполнения запроса с B+tree индексами в СУБД MongoDB.

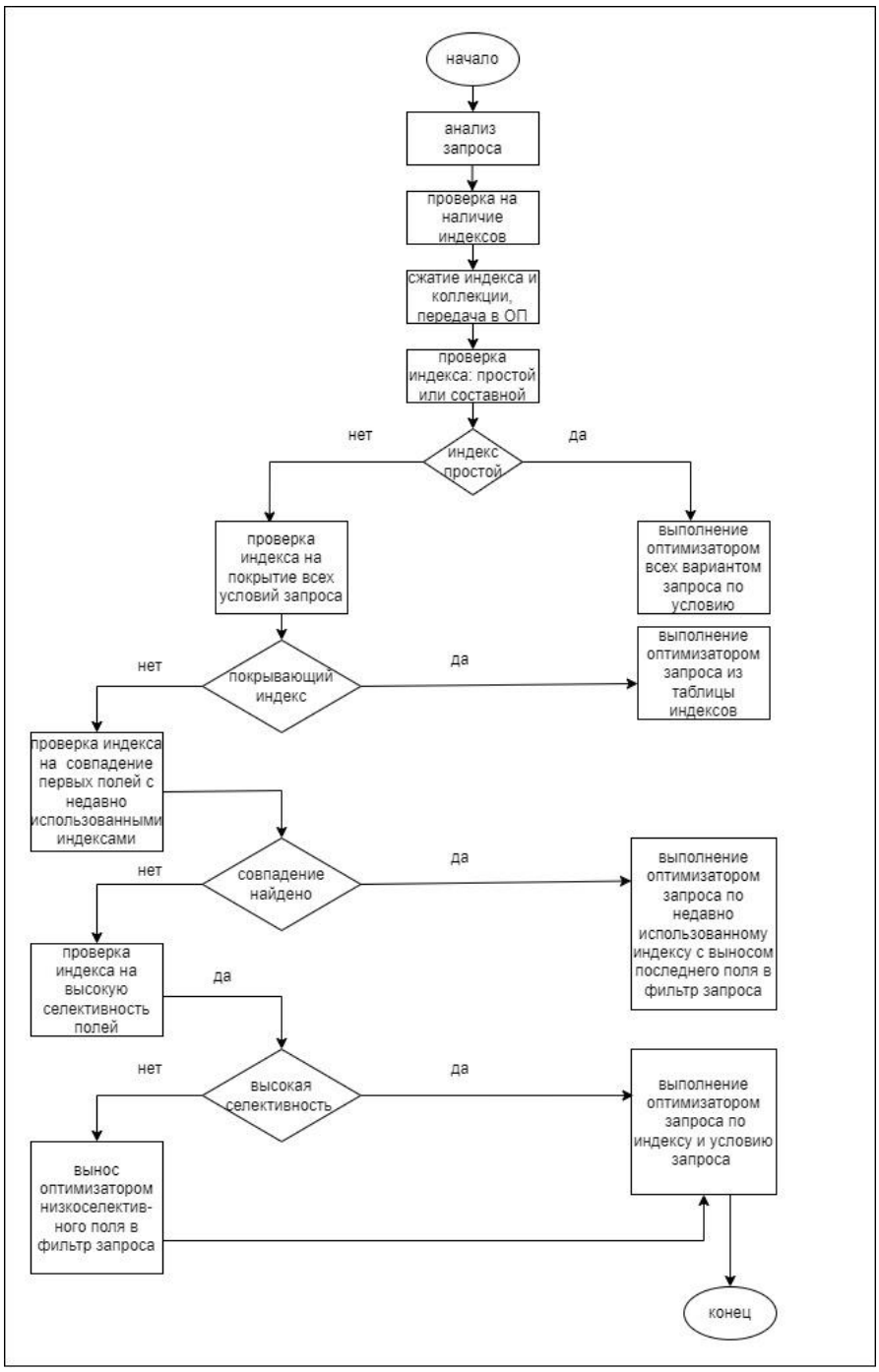


Рисунок 24 – Логическая схема выбора оптимизатором плана выполнения запроса с B-tree индексами в СУБД MongoDB

2.3.3 Исследование факторов оптимизации сложных запросов к таблицам/коллекциям с большим количеством записей

Почему мы выбрали оптимизацию запросов с использованием B-tree индексов и отказались от применения полнотекстового поиска по лексемам с

помощью GIN и GiST индексов? Ответ содержит особенности в структуре индексов, принципах их работы и обновления:

1) GIN и GiST индексы предназначены для поиска в больших текстах с учетом морфологии и синонимов; B-tree индексы используют фильтрацию и сортировку метаданных, эффективно работают с метаданными, которые предоставляет пользователь в строке поиска документа;

2) при очень больших количествах записей TID могут быть длинными, значит, будет увеличиваться время их чтения с диска. Также при частых лексемах происходит частое обращение к диску, что значительно снижает скорость запроса в итоге. При использовании B-tree индексов, соответствие индекса целой строке в таблице снижает количество обращений к таблице на диске;

3) при работе с таблицами с большим количеством записей таблицы GIN и GiST индексов разбухают сильно быстрее, чем таблицы B-tree индексов, таким образом, значительно увеличивается время уже на этапе чтения таблиц GIN и GiST индексов;

4) семантический поиск по тексту аннотации к статье с применением SBERT — двунаправленные кодирующие представления предложений на основе трансформаторов - мы рассмотрим в следующем пункте.

Выбор PostgreSQL и MongoDB обусловлен областью исследования в автоматизированной научно-технической библиотеке. Обе СУБД предназначены для хранения и операций с документами.

При увеличении количества записей в таблицах/коллекциях до определенного предела, когда длительность отклика системы на действия пользователя достигает критических значений, встает вопрос выбора между горизонтальным масштабированием или пересмотром других возможностей оттянуть этот момент. Сложность настройки и поддержки, рост материальных затрат, а также риски снижения производительности системы заставляют искать другие варианты повышения ее производительности. Среди таких способов анализ и переписывание медленных запросов, также снижение

количества соединений JOIN для ускорения чтения, оптимизация индексов, анализ плана запросов.

В нашем исследовании мы выбрали анализ плана запросов, как основной метод исследования оптимизации сложных запросов к таблицам и коллекциям с большим количеством записей.

Также, исходя из данных информационных моделей, мы будем изучать факторы снижения скорости запросов в обеих СУБД:

1) селективность;

2) фильтрация NULL значений в запросе;

3) покрытие условия запроса.

Показателем оптимизации сложных запросов к таблицам и коллекциям с большим количеством записей будем рассматривать время выполнения этих запросов и сравнение этого времени со временем первых трех фаз зрительного анализатора человека-оператора при восприятии зрительной информации. Таким образом, мы стремимся достичь эргономичности наших запросов.

2.4 Исследование возможности предоставления пользователю нового вида поиска, основанный на векторном пространстве признаков литературного источника

Семантический поиск документа по пространству признаков представляет новое поколение поиска [31,32,33].

Само название Sentence Bidirectional Encoder Representations from Transformers дает надежду на реализацию семантического поиска, поиска по смыслу предложения. Это значительно отличается от традиционного поиска по совпадению ключевых слов.

Особенностями всех моделей класса BERT чтение текста в обе стороны от каждого слова в предложении. Также, 12 голов внимания, либо 16 голов внимания повышают качество поиска, максимально приближая его к поиску по смыслу.

Что ожидаем от этого семантического поиска:

1) исключение влияния некорректного ввода информации, ошибки символа или слова на качество поиска и выдачу релевантных документов по запросу;

2) понимание контекста, слов-синонимов в этом контексте, такого не могут делать другие виды поиска, значит, повышение точности поиска;

3) выявление связей между общими и специализированными понятиями, значит, расширение объема возвращаемых релевантных документов;

4) улучшение семантического поиска за счет обучения модели.

Выбор конкретной модели из всех представителей класса BERT проходил по следующим показателям:

1) количество языков, представленных в предобученной модели. Мы выбрали один английский язык, все статьи, с которыми работали написаны на нем. Таким образом, значительно сократили требуемый объем памяти и время обучения;

2) учет верхнего и нижнего регистра в тексте документов, для нашего исследования важно, научно-техническая литература включает множество названий и имен собственных;

3) скорость обучения под конкретные задачи поиска научно-технической литературы в математической и информационной области знаний.

Выбор остановили на модели SBERT_all-mpnet-base-v2 (Masked and Permuted Pre-training for Language Understanding). Ее достоинства:

1) применяется для семантического поиска, кластеризации, сравнения текстов;

2) маскирует токены и перестанавливает, меняя их порядок, чтобы повысить точность семантического поиска;

3) создает векторы признаков для каждого документа;

4) для получения векторов признаков используется mean pooling с учётом маски внимания (бинарный вектор: 1 - реальный токен, 0 - паддинг; исключаются токены паддинга путем умножения эмбеддинга каждого токена

на соответствующее значение маски; суммируются все значения реальных токенов и делятся на их количество); это позволяет сохранить контекст от влияния пустых токенов;

5) эти векторы имеют большую плотность потому, что с помощью функции потерь минимизирует косинусные расстояния между эмбедднгами близких по смыслу документов и максимизирует косинусное расстояние между эмбеддингами слабо связанных по смыслу документов; это повышает качество поиска, исключая ложные совпадения.

Недостатки модели BERT, которые выяснились в предварительном эксперименте:

1) на узкоспециализированных и близких по тематике областях знаний модель может кластеризовать неразмеченные данные на меньшее количество классов;

2) вместо четырех предложенных классов, модель увидела с наибольшей вероятностью 2-3 класса;

3) при классификации на четыре класса, точность классификации снижается.

Таким образом, требуется дообучение модели на этих данных. Также, мы пришли к выводу, что обучение с учителем по размеченным данным принесет лучшие результаты кластеризации и создания эмбеддингов. На рисунке 25 представлены результаты выбора моделью оптимального числа кластеров по созданным эмбеддингам для 409 статей по 4 областям знаний: SQL-запросы, качество программного обеспечения, управление знаниями и анализ временных рядов:



Рисунок 25 — результаты выбора моделью оптимального числа кластеров по созданным эмбеддингам для 409 статей по 4 областям знаний

На рисунке 26 представлена визуализация кластеров статей.

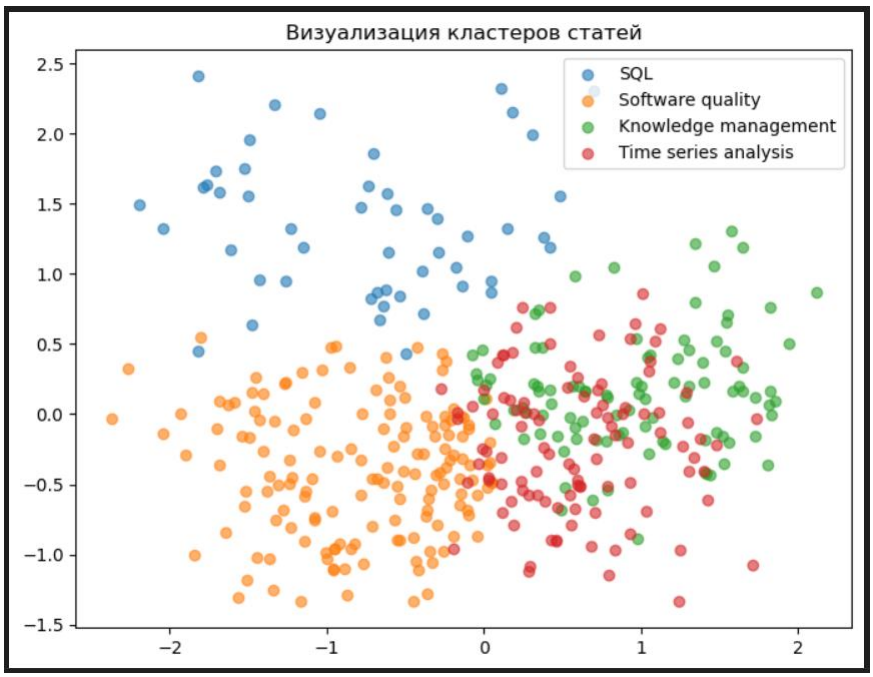


Рисунок 26 — Визуализация кластеров статей

Как видно на рисунке 26 эмбеддинги векторов признаков документов по анализу временных рядов находятся в тесном соседстве с эмбеддингами признаков документов по управлению знаниями. Это объясняем тем, что в менеджменте по управлению знаниями анализ временных рядов является основным инструментом. Таким образом, получен отрицательный опыт:

1) выбора исходных документов по близким в семантическом смысле областям знаний;

2) неправильный выбор модели bert_base_cased для семантического поиска в узкоспециализированных областях знания;

3) обучения без разметки исходных данных на модели bert_base_cased, которая предобучена на широком круге тематик.

Для создания эмбеддингов и вычисления косинусного сходства между ними в основном эксперименте были приняты следующие меры:

1) выбраны классы непересекающихся областей знания (NLP, Software quality, Knowledge management);

2) выбрана модель для более сильного семантического поиска SBERT_all-mpnet-base-v2;

3) разметка исходных данных не требуется потому, что вычисляются косинусные расстояния между эмбеддингами и по этому косинусному сходству находятся близкие по смыслу документы;

4) для очистки данных в «.csv» файле созданы поля: название статьи, авторы, аннотация. Для создания эмбеддингов использовались только аннотации потому, что информация из других полей не является полезной для обучения.

Таким образом, были получены эмбеддинги по трем классам статей, взятых из arXiv. Также были получены эмбеддинги массива запросов от пользователя.

В таблице 3 представлены результаты косинусного сходства в эмбеддингах запросов пользователя и статей в базе данных, выводится по пять релевантных статей для каждого запроса.

Таблица 3 — Значения наилучших оценок косинусного сходства пяти статей

№ запроса	Статья 1	Статья 2	Статья 3	Статья 4	Статья 5
1	0,8297	0,5903	0,5217	0,5156	0,4990
2	0,7666	0,6581	0,5817	0,5526	0,4929
3	0,5051	0,4621	0,4053	0,3867	0,3634

№ запроса	Статья 1	Статья 2	Статья 3	Статья 4	Статья 5
4	0,7620	0,7112	0,6737	0,6334	0,6083
5	0,7284	0,4676	0,4500	0,4132	0,3970
6	0,8780	0,5947	0,4979	0,4826	0,4443
7	0,4957	0,4717	0,4647	0,4551	0,4539
8	0,4654	0,4636	0,4166	0,4069	0,4021
9	0,8322	0,6837	0,6239	0,5999	0,5741
10	0,6197	0,5177	0,5026	0,5024	0,4895

Как видно из представленных данных косинусное сходство по запросам пользователя варьирует от 0.8780 до 0.3634.

В результате изученных литературных источников [34-39] были определены эмпирические значения оценок косинусных расстояний в эмбеддингах сравниваемых между собой документов. В данном исследовании мы также придерживались этих оценок косинусных расстояний в эмбеддингах найденных документов с эмбеддингами запросов пользователя

В таблице 4 представлены значения оценок косинусных расстояний в эмбеддингах найденных документов с эмбеддингами запросов пользователя.

Таблица 4 — Значения оценок косинусных расстояний в эмбеддингах найденных документов с эмбеддингами запросов пользователя.

Значение косинусного расстояния	Оценка	Количество
0.8780 - 0.7500	высокое	5
0.7500 - 0.40000	среднее	48
0.45000 - 0.2833	низкое	4

Вывод:

Разработана архитектура автоматизированной научно-технической библиотеки в соответствии с ее функциональными требованиями. Для изучения вариантов повышения скорости выполнения запросов пользователей разработаны информационные модели сложных запросов к таблицам и коллекциям с большим количеством записей. В архитектуру системы включены

компоненты семантического и расширенного поиска. Создана база научных статей по трем областям знания. Создан файл с вариантами запросов пользователя. В результате отбора была определена модель SBERT_all-mpnet-base-v2 под задачи семантического поиска в конкретных трех областях знания.

В результате дообучения получены эмбеддинги векторов признаков для статей в базе данных и эмбеддинги векторов признаков по запросам пользователя. Определена схема сравнения полученных эмбеддингов по косинусному сходству. Определена экспертная система оценок качества семантического поиска используя результаты работ других исследователей.

3 РЕЗУЛЬТАТЫ ИССЛЕДОВАНИЯ И РАЗРАБОТКИ АВТОМАТИЗИРОВАННОЙ СИСТЕМЫ НАУЧНО-ТЕХНИЧЕСКОЙ БИБЛИОТЕКИ

По созданной архитектуре автоматизированной научно-технической библиотеки разработано программное приложение, написанное и протестированное на языке Java. Также на языке Python был написан сервис, обрабатывающий запросы пользователя и отвечающий за работу модели.

Проблема скорости выполнения SQL-запросов связана с эргономичностью сложных запросов к таблицам и коллекциям с большим количеством записей. Время ожидания ответа на запрос расценивалось в пределах времени первых трех фаз восприятия светового сигнала зрительным анализатором человека. Такое время соответствует примерно 1,5 сек. Время выполнения запросов, превышающее эту временную границу, расценивалось, как неудовлетворительное. На время выполнения запросов влияет множество факторов: индексы, объем данных, выбор плана выполнения запроса оптимизатором, текущая нагрузка на базу данных, задержки на уровне сети.

Необходимость оптимизации запросов заключается также в снижении нагрузки на СУБД. От структуры составленного запроса зависит, какое количество вычислительных ресурсов (оперативная память, процессорное

время) будет использоваться. В данной работе на основе разработанных информационных моделей сложных поисковых запросов с B-tree индексами на примерах PostgreSQL и MongoDB были выбраны факторы, влияющие на скорость выполнения таких запросов.

3.1 Исследование факторов снижения скорости запросов пользователя

3.1.1 Исходные данные

Эксперимент проводился на сгенерированных данных онлайн научной библиотеки. Созданы таблицы в PostgreSQL: «resource» с 70 млн записей, «article» с 30 млн записей и «userlib» с 30 млн записей. В MongoDB созданы коллекция «resource» с 23 млн документов. Выполнялись сложные запросы к двум таблицам в PostgreSQL и сложные запросы ко вложенному полю в документе коллекции MongoDB. Был изучен план выполнения запросов, возвращаемый планировщиком. Изучались особенности применения B-tree индексов в этих запросах для каждой СУБД.

Ресурсы научной онлайн библиотеки состоят из нескольких типов: статьи, книги, презентации, видео, методические указания. В таблицах PostgreSQL у этих типов ресурсов есть одинаковые поля: название источника, автор, тип источника, путь к файлу, год издания. Таблица «resource» содержит поля «id» - идентификационный номер, «name» - название источника, «author» - автор, «type» - тип источника, «path» - путь к файлу, «year» - год издания. Ресурс является суперклассом для статьи, видео и т.д. Таблица «article» включает поля: «id», «DOI», «ISBN» и «resource_id» для связи с суперклассом «resource». Аналогично связаны другие классы видео, презентация, научный том и методические указания с суперклассом ресурс через поле «resource_id». Таблица «userLibrs» включает в себя поля «id», «name» - имя пользователя, «rights» - права_пользователя и «resource_id» для связи с суперклассом «resource».

В коллекции MongoDB собраны документы разного типа с разными полями; статья с полями «_id», «name» - название источника, «author» - автор, «type» - тип источника, «path» - путь к файлу, «year» - год издания, «DOI», «ISBN» и другие типы ресурсов со своими полями. Количество полей у разных типов документов было разное.

В PostgreSQL выполнялись запросы с B-tree индексами к двум таблицам «resource» и «article», «resource» и «userLib».

В MongoDB выполнялись запросы с B+tree индексами к вложенному низкоселективному полю «language» документов типа «scientificVolume», вложенному высокоселективному «DOI» и низкоселективному полю «ISBN» документов типа «article»; также проводились запросы с покрывающими индексами, где явно указывались «_id».

3.1.2 Методы

Целью оптимизации поисковых запросов к отношениям/коллекциям с большим количеством записей было снижение времени выполнения запросов с B-tree индексами до времени первых трех фаз зрительного анализатора человека-оператора.

Методами взвешивания таблиц были выбраны:

1) `pg_size_pretty(pg_relation_size('userLibr_2'))`,

2) обращение к статистике метатаблиц:

3) `select * from bt_metap('index_btree_us_2')`,

4) `bt_page_stats('index_btree_us_2', 3)`.

Методами анализа запросов были выбраны:

1) `EXPLAIN ANALYZE` для запросов в PostgreSQL,

2) `explain("executionStats")` для запросов в MongoDB.

3.1.3 Результаты исследования и их обсуждение

В таблице 5 представлены данные по количеству записей в таблицах PostgreSQL и коллекции «resource» MongoDB и время полного считывания в сложных запросах (запросы №№ 1, 2).

Таблица 5 — Данные по количеству записей в таблицах PostgreSQL и коллекции «resource» MongoDB и время полного считывания в сложных запросах

СУБД	PostgreSQL	MongoDB
Объём таблиц/коллекций (количество записей)	«resource»: 70 000 000; «article»: 30 000 000; «userlab»: 30 000 000	«resource»: 23 000 000;
Время полного сканирования таблиц/коллекции (мс)	92683,329	28326

В таблице 6 представлены результаты выполнения сложных запросов в PostgreSQL и MongoDB.

Таблица 6 — Результаты выполнения сложных запросов в PostgreSQL и MongoDB

	PostgreSQL			MongoDB		
	№ запроса	Время, (мс)	Снижение времени, %	№ запроса	Время, (мс)	Снижение времени, %
Селективность	4	9,74	99,99	14	83	99,71
	3	5,06	99,99	15	77	99,73
Фильтрация NULL-значений в запросе	6	2,88	99,99	-	-	-
	5	3,17	99,99	-	-	-
Покрытие условия	9	5924,29	93,61	12	118	99,58
	10	6,12	99,99	13	72	99,75
Серия сложных запросов	[9, 10, 3, 4]	[5924,29, 6,2, 5,7, 9,48]	->99,99	[12, 13, 15, 14]	[118, 9, 2, 0]	->100

В запросе №1 происходит полное считывание из двух таблиц «resource» и «article», время выполнения 92683.329 мс.

В запросе №2 происходит полное считывание из коллекции «resource» до вложенного уровня включительно, время выполнения 28326 мс.

В запросе №3 присваиваем пользователю с именем «user2000» право «admin», повышая селективность поля максимально и выполняем запрос по индексу B-tree к двум таблицам «resource» и «userlibr». Получаем план выполнения запроса:

```
Index Scan using index_btree_resource on resource r
Index Scan using index_btree_us on userlibr u
Execution Time: 4.351 мс
```

В запросе №5 к двум таблицам «resource» и «article» по индексам B-tree оптимизатор при выборе плана по индексам будет читать все строки таблицы индексов с учетом значений null.

В запросе №6 создаем индекс с учетом null значений:

```
CREATE INDEX index_btree_article_doi_non
    ON article
    USING BTREE(DOI)
    WHERE DOI IS NOT NULL;
```

В запросе №9 к таблицам «resource» и «article» планировщик выбирает план чтения по индексам в таблице «resource» и полного сканирования таблицы «article» по причине отсутствия индекса для этой таблицы.

В запросе №10 пример покрывающих индексов, когда происходит чтение из таблиц индексов для обеих таблиц «resource» и «article», не происходит обращение к самим таблицам.

В запросе №12 по индексу B+tree к вложенному полю коллекции «resource» происходит чтение таблицы индексов и чтение одного документа:

```
executionStats: {
    executionSuccess: true,
    nReturned: 1,
```

```
        executionTimeMillis: 118,
        totalKeysExamined: 1,
        totalDocsExamined: 1,
```

В запросе №13 по индексу B+tree к вложенному полю коллекции «resource» происходит чтение только таблицы индексов и не происходит обращение к таблице, т.е. пример покрывающих индексов:

```
executionStats: {
    executionSuccess: true,
    nReturned: 1,
    executionTimeMillis: 72,
    totalKeysExamined: 1,
    totalDocsExamined: 0,
    executionStages: {
        stage: 'PROJECTION_COVERED',
        nReturned: 1,
```

Это получилось достигнуть с помощью явного указания «_id = 0»:

```
db.resource.explain('executionStats').find({
    name: 'name21200000',
    author:  'author21200000',
    'res_description.language':'Russian'
}, {
    name: 1,
    "_id": 0,
    author:1,
    'res_description.language': 1,
});
```

В запросе 14 по индексу B+tree с низкоселективным полем ISBN к вложенному полю планировщик использует вложенное поле документа в индексе, как фильтр запроса:

```
winningPlan: {
```

```
        stage: 'FETCH',
        filter: {
            'res_description.ISBN': {
                '$eq': 'ISBN_500'
            }
        },
        inputStage: {stage: 'IXSCAN',
```

и проводит поиск по имеющемуся в памяти ключу, первые поля которого совпадают с созданным нами ключом для этого запроса:

```
    direction: 'forward', indexBounds: {
        name: ['["name23000000", "name23000000"]'],
        author: ['["author23000000", "author23000000"]'],
        Language: ['[MinKey, MaxKey]']
    }
```

Выполнение запроса:

```
    executionStats: {
        executionSuccess: true,
        nReturned: 1,
        executionTimeMillis: 83,
        totalKeysExamined: 1,
        totalDocsExamined: 1,
        executionStages: {
            stage: 'FETCH',
            filter: {'res_description.ISBN':
                {'$eq': 'ISBN_500'}
            }
        }
    }
```

В запросе 15 по индексу B+tree с высокоселективным полем DOI к вложенному полю оптимизатор также использует вложенное поле документа в индексе, как фильтр запроса:

```
winningPlan: {
    stage: 'FETCH',
    filter: {
        'res_description.DOI:{
            '$eq': 'DOI_23000000'
        }
    },
    inputStage: {
        stage: 'IXSCAN',
```

и проводит поиск по имеющемуся в памяти ключу, первые поля которого совпадают с созданным нами ключом для этого запроса:

```
indexBounds: {
    name: [ '["name23000000", "name23000000"]'],
   author: ['["author23000000", "author23000000"]'],
    'res_description.language':['[MinKey, MaxKey]'}…
```

Выполнение запроса:

```
executionStats: {
    executionSuccess: true,
    nReturned: 1,
    executionTimeMillis: 77,
    totalKeysExamined: 1,
    totalDocsExamined: 1,
```

Таким образом, оптимизатор выносит вложенное поле документа в фильтр плана выполнения запроса, использует частично подходящий имеющийся в кэше индекс по другим полям индекса.

Для выявления времени сжатия и передачи данных в память подсистемой хранения WiredTiger проведена серия запросов. Сравнивалось время

выполнения первого запроса из серии запросов и время выполнения этого же запроса после очищения буфера, повторного создания индекса на эту коллекцию и выполнения серии запросов. Выяснилось, что время выполнения одного и того же запроса сильно сократилось с 118 мс до 22 мс в результате предварительного запроса на создание индекса на эту коллекцию. В условиях приближения к чистому эксперименту можно считать разницу во времени выполнения этих запросов временем работы WiredTiger.

Аналогичная серия запросов была проведена и в PostgreSQL. Сравнивалось время выполнения запросов в режиме после очищения буфера и в режиме серии запросов. Как видно из таблицы 2, время одиночных запросов очень близко к времени выполнения этих запросов в серии запросов для PostgreSQL. В то время, как для MongoDB время выполнения запросов в режиме серии запросов значительно сокращается по сравнению с временем выполнения этих запросов в одиночном режиме после очищения буфера. Это время в серии запросов стремиться к нулю даже при низкоселективном поле в фильтре запроса (см. Таблицу 2). Такое наблюдение является подтверждением, что в результате сжатия вся коллекция находится в оперативной памяти. Этот пример позволяет убедится, что для MongoDB рациональным будет выполнение серии запросов к большой коллекции по сравнению с одиночными запросами. Для PostgreSQL проведение одиночных запросов или серии запросов не влияет на время их проведения.

Время зрительного анализатора человека-оператора можно рассматривать, как «время зрительной фиксации (для простых фигур – 0,2 сек и для ознакомления с ситуацией – 0,65 сек) $t_{фикс}$, время сохранения зрительного ощущения (инерция зрения) $t_{ин}$ и время зрительного восприятия $t_{воспр}$. Это неполный набор ЗА ЧО» [17]. Нас интересуют первые перечисленные три фазы.

В нашем эксперименте время сложного запроса на чтение без индексов к отношению resource с 70 млн записей составило 92,683 сек. Это значительно превышает время зрительного анализатора человека-оператора.

Результаты эксперименте вычисления времени выполнения сложного запроса на чтение с B-tree индексами к отношению resource с 70 млн записей в PostgreSQL указаны в таблице 7.

Таблица 7 — Время выполнения сложных запросов на чтение в PostgreSQL

Состояние отношения	Время выполнения, (мс)
с применением селективности	5,06
с учетом значений IS NOT NULL в индексе	2,88
с покрывающими индексами	6,12

Результаты эксперименте вычисления времени выполнения сложного запроса на чтение с B+tree индексами к отношению resource с 23 млн записей в MongoDB указаны в таблице 8.

Таблица 8 — Время выполнения сложных запросов на чтение в MongoDB

Состояние отношения	Время выполнения, (мс)
с применением селективности	77
с покрывающими индексами	72

Все значения меньше времени зрительного анализатора человека-оператора (первые три фазы). Так мы доказали эргономичность наших запросов.

Таким образом:

1) применение индексов B-tree в сложных запросах к отношениям или коллекциям с большим количеством записей в PostgreSQL и MongoDB сокращает время запроса на 99 %;

2) для PostgreSQL наиболее информативными для снижения времени сложных запросов к таблицам с большим количеством записей оказались факторы применения в запросах покрывающих индексов, высокой селективности полей индексов, явное указание значений NULL в таблице индексов;

3) для запросов с B-tree индексами к таблицам PostgreSQL важен порядок полей в индексе; уникальные и высокоселективные поля рационально прописывать на первой позиции;

4) для MongoDB наиболее информативным выявлен фактор кэширования в оперативную память таблицы индексов и таблицы данных, к которым на текущий момент происходит обращение. Поэтому, важно выполнять запросы серий запросов к одной коллекции;

5) при выполнении серии сложных запросов к одной коллекции время работы WiredTiger прибавляется ко времени первого запроса в серии запросов к одной коллекции; время следующих запросов в серии сокращается и стремится к нулю;

6) при выполнении серии сложных запросов с B-tree индексами первым запросом актуально выполнять самый короткий запрос с учетом покрывающих индексов и селективности; это сократит время всей серии запросов;

7) при выполнении выше перечисленных условий сложных запросов с B-tree индексами, время выполнения не превышает времени первых трех фаз восприятия светового сигнала зрительным анализатором человека-оператора. Поэтому мы можем утверждать, что такие запросы являются эргономичными.

3.2 Исследование возможностей семантического поиска в автоматизированной научно-технической библиотеке

Современные модели для создания векторных представлений текстов, такие как SBERT_all-mpnet-base-v2, показывают хорошие результаты в задачах поиска и сопоставления смыслового содержания. Однако для повышения точности в конкретных предметных областях требуется дополнительное обучение модели с учётом специфики этих тем. В данной главе описывается процесс адаптации модели на примере тематик NLP, Software quality и Knowledge management, а также анализируются результаты и возможности её применения.

3.2.1 Сбор и подготовка тематических данных

Для проведения дообучения был сформирован специализированный корпус текстов, включающий материалы по NLP, Software Quality и Knowledge Management. Источники включали научные публикации, технические документы и отраслевые отчёты, суммарным объёмом 310 текстов.

Перед использованием данные прошли этап очистки: удаление лишних символов и тегов, перевод в нижний регистр, а также лемматизация и токенизация с помощью современных NLP-инструментов. Были исключены стоп-слова и нерелевантные элементы.

Данные были разделены на обучающую, валидационную и тестовую выборки в пропорции 70:15:15, сохраняя равномерное распределение по тематикам.

3.2.2 Процесс дообучения модели

Дообучение осуществлялось через fine-tuning с использованием задачи оценки семантического сходства между парами текстов.

Основные параметры обучения включали:

Оптимизатор AdamW с learning rate 2e-5;

Размер батча 32;

Число эпох −4;

Функция потерь −CosineSimilarityLoss, направленная на максимизацию косинусного сходства.

Обучение выполнялось на GPU с использованием библиотек Hugging Face Transformers и Sentence-Transformers.

3.2.3 Анализ результатов

Качество модели оценивалось по косинусному сходству и метрике MSE.

Косинусное сходство применялось для кластеризации и классификации: для оценки близости объектов в пространстве признаков, для сравнения семантической близости текстов при поиске по запросу пользователя.

Косинусное сходство измеряет косинус угла между двумя ненулевыми векторами A и B по формуле 3.

$$cosine_similarity(A, B) = \frac{A * B}{||A|| * ||B||}, \quad (3)$$

где A * B — скалярное произведение векторов,

||A|| и ||B|| — нормы (длины) векторов,

результат — число от −1 до 1, где 1 означает, что векторы направлены одинаково (максимальное сходство), 0 — что векторы ортогональны (нет сходства), -1 — что векторы направлены в противоположные стороны.

Инвариантность к масштабу: косинусное сходство учитывает только угол между векторами, а не их длину. Это значит, что два документа с разной длиной, но схожим содержанием, будут иметь высокое сходство.

Применяя данные таблицы 3, мы получаем преимущественно среднюю оценку по косинусному сходству поиска литературных источников с помощью предобученной модели SBERT_all-mpnet-base-v2. Полученные результаты демонстрируют среднее качество семантического поиска, модель пригодна к практическому применению, требуется дообучение модели на специализированных текстах по требуемой тематике.

Также качество семантического поиска модели SBERT_all-mpnet-base-v2 оценивалось метрикой MSE по формуле 4.

$$MSE = \frac{1}{n}\sum_{i=1}^{n}(y_i - \hat{y}_i)^2, \quad (4)$$

где n — число наблюдений,

y_i — фактическое значение,

$\hat{y}_i$ — предсказанное моделью значение.

Метрика чувствительна к выбросам. Присутствие шума в реальных данных не позволяет добиться идеальных значений метрики, когда она равна 0. Чем ближе к 0 полученные значения MSE, тем лучше предсказания модели совпадают с реальными данными, тем лучше она решает задачи классификации и семантического поиска литературных источников по запросу пользователя. Так как метрика чувствительна к выбросам, то добиваясь хороших значений, необходимо тщательно подготовить исходные данные.

В нашем исследовании были получены следующие значения MSE для каждого из 10 запросов пользователя. В таблице 9 представлены вычисления косинусного сходства и результаты MSE по поиску 5 самых релевантных статей для каждого запроса пользователя.

Таблица 9 — результаты MSE по поиску 5 самых релевантных статей для каждого запроса пользователя.

№ запроса	MSE
1	-0,2049
2	-0,1908
3	-0,2401
4	-0,1968
5	-0,2271
6	-0,1961
7	-0,2091
8	-0,2386
9	-0,2334
10	-0,1996

В таблице 10 представлены оценки значений MSE.

Таблица 10 — Оценки значений MSE

Значение negative MSE	Качество модели	Описание
От -0.05 до 0	Отличное	Очень низкая ошибка, модель предсказывает почти идеально
От -0.10 до -0.05	Хорошее	Низкая ошибка, модель показывает высокую точность
От -0.20 до -0.10	Среднее	Приемлемая ошибка, модель работает достаточно хорошо
Меньше -0.20 (например, -0.3 и ниже)	Плохое	Высокая ошибка, модель нуждается в улучшении

Сравнение результатов, полученных MSE с табличными оценками, позволяет сделать вывод, что модель работает достаточно хорошо.

Качество семантического поиска для полученной модели также оценивалось метриками: Accuracy, Precision, Recall, F1-мера. В таблице 11 представлены результаты оценки семантического поиска для полученной модели.

Таблица 11 — результаты оценки семантического поиска полученной модели.

Метрика	Значение
Accuracy	0,7236
Precision	0,4067
Recall	0,6375
F1-мера	0,4966

Таким образом, полученные оценки качества семантического поиска для дообученной модели SBERT_all-mpnet-base-v2 по косинусному сходству, по MSE, Accuracy, Precision, Recall, F1-мера сходятся в своих значениях. Модель достаточно хорошо выполняет семантический поиск по запросу пользователя и пригодна для применения.

Выводы:

На основе разработанной архитектуры автоматизированной научно-технической библиотеки были проведены эксперименты и достигнуты

результаты эргономического показателя времени выполнения сложных запросов пользователя в традиционном поиске по метаданным литературного источника.

Также проведен эксперимент по применению нового вида семантического поиска в автоматизированной научно-технической библиотеке. Дообучена модель SBERT_all-mpnet-base-v2 под конктретные данные из трех областей знания, получены вектора признаков всех статей созданной базы данных и вектора признаков предполагаемых запросов пользователя. Проведено сравнение по косинусному сходству этих пространств признаков. Протестировано качество семантического поиска по метрикам косинусного сходства, MSE, Accuracy, Precision, Recall, F-меры. Результаты тестирования выявили сходство оценок по всем метрикам качества семантического поиска. Дообученная модель SBERT_all-mpnet-base-v2 показывает достаточно хорошее качество семантического поиска в означенных областях знания и пригодна к применению.

В данном исследовании был изучен еще один вид расширенного поиска через интеграцию с информационной платформой arXiv. Такая интеграция с платформой arXiv предоставляет пользователю быстрый доступ к большому количеству статей, отсортированных по дате издания. Однако поиск в arXiv осуществляется по совпадению и при наличии в современных статьях множества одинаковых аббревиатур с разной расшифровкой, а также часто встречающихся терминов во всех областях знания снижают релевантность возвращаемых статей.

ЗАКЛЮЧЕНИЕ

В данной работе предложен новый подход в улучшении и расширению функционала автоматизированной научно-технической библиотеки:

- в снижении времени выполнения сложных запросов и уход от горизонтального масштабирования базы данных,

- в расширении возможностей поиска за счет интегрирования с информационными платформами,

- в семантическом поиске по свободному контексту запроса пользователя, что возвращает всегда релевантные документы даже при некоторых ошибках в запросе.

Была достигнута оптимизация времени выполнения сложных запросов и достижение показателей эргономичности запроса, т.е. не превышение времени 1,5сек для получения ответа пользователем. Выявлены положительные особенности хранения информации в PostgreSQL для составления определенного вида запросов к таблицам с большим количеством записей. Также выявлены положительные особенности хранения информации в MongoDB для составления другого типа сложных запросов к коллекциям с большим количеством записей.

Было предложено расширить возможности поиска пользователя с помощью интеграции с информационной платформой arXiv. Такой опыт интеграции можно использовать в других научно-технических библиотеках, а также в поисковых системах корпоративных фирм.

Было достигнута реализация нового поиска, семантического поиска, поиска не по совпадению слов, но по косинусному сходству в эмбеддингах; эмбеддинги были сформированы по пространству признаков каждого документа. Данный вид поиска хорошо применим для научно-технических библиотек с узкоспециализированной тематикой в определенных областях знания. Для широкого круга пользователей этот вид поиска также актуален. В

таком поиске ошибки при некорректном вводе информации пропускаются системой и модель ищет по вектору признаков введенного контекста.

Возможности разработанной научно-технической библиотеки представляют практический интерес для привлечения и удержания пользователей.

СПИСОК ИСПОЛЬЗОВАННЫХ ИСТОЧНИКОВ

1. Мартин Р. Чистая архитектура. Искусство разработки программного обеспечения. — М., СПб.: Питер, 2024. — 352 с.

2. Официальный сайт Гуманитарного университета, [Электронный ресурс]. — URL https://gu-ural.ru/education/library/internet_resources/library-online/ (дата обращения 10.10.2024).

3. Официальный сайт Библиоклуб.ру, [Электронный ресурс]. — URL https://biblioclub.ru/ (дата обращения 10.10.2024).

4. Официальный сайт Znanium [Электронный ресурс]. — URL https://znanium.ru/ (дата обращения 10.10.2024).

5. Официальный сайт Национальная электронная библиотека, [Электронный ресурс]. — URL https://rusneb.ru (дата обращения 10.10.2024).

6. Официальный сайт Мурманский арктический университет, [Электронный ресурс]. — URL https://www.mauniver.ru/ (дата обращения 10.10.2024).

7. Официальный сайт Вятский государственный университет, [Электронный ресурс]. — URL https://www.vyatsu.ru/biblioteka-3/elektronnaya-biblioteka/elektronnyie-resursyi/knigi-1.html#neb (дата обращения 10.10.2024).

8. Официальный сайт электронно-библиотечная система book, [Электронный ресурс]. — URL https://book.ru (дата обращения 10.10.2024).

9. Официальный сайт academia-moscow, [Электронный ресурс]. — URL academia-moscow.ru (дата обращения 11.10.2024).

10. Официальный сайт юрайт, [Электронный ресурс]. — URL https://urait.ru (дата обращения 11.10.2024).

11. Официальный сайт лань, [Электронный ресурс]. — URL https://e.lanbook.com/ (дата обращения 11.10.2024).

12. Елисеева Е.А., Горякин Б.С., Виноградова М.В., Черненький М.В. Оценка времени выполнения поисковых запросов в nosql и объектно-

реляционной базах данных, [Электронный ресурс]. — URL https://radiotecru.translate.goog/en/journal/Dynamics_of_Difficult_Systems–XXI_century/number/2022/2/article/21695?_x_tr_sch=http&_x_tr_sl=auto&_x_tr_tl=ru&_x_tr_hl=ru (дата обращения 18.10.2024).

13. Кирилл Мухин. OTUS IT Отлайн - образование -B-tree индексы в PostgreSQL//Курс «PostgreSQL для администраторов баз данных и разработчиков», [Электронный ресурс]. — URL https://www.youtube.com/watch?v=FfZGZreAxcc&t=3076s (дата обращения 21.10.2024).

14. Позняк А.А., Горячкин Б.С. Оценка времени выполнения SQL-запросов в озерах данных // Наукосфера. – 2024. — №5-1. — С. 60-70.

15. 14 questions about indexes in SQL Server that you were embarrassed to ask when parsing the end of the file, [Электронный ресурс]. — URL https://habr.com/ru/articles/247373/ (дата обращения 24.10.2024)

16. Индексы в PostgreSQL-1, [Электронный ресурс]. — URL https://habr.com/ru/companies/postgrespro/articles/326096/ (дата обращения 24.10.2024).

17. Типы индексов, [Электронный ресурс]. — URL https://postgrespro.ru/docs/postgresql/14/indexes-types (дата обращения 24.10.2024)

18. Горячкин Б.С. Проектная эргономика автоматизированных систем обработки отображения информации и управления: монография / Б. С. Горячкин. Т. 2. — Москва: Спутник+, 2023-, 2023. — 496 с.; ISBN 978-5-9973-6782-4.

19. Елисеева Е.А., Горячкин Б.С., Виноградова М.В., Исследование производительности СУБД при работе с кластерными базами данных на основе эргономического анализа, [Электронный ресурс]. — URL https://cyberleninka.ru/article/n/issledovanie-proizvoditelnosti-subd-pri-rabote-s-

klasternymi-bazami-dannyh-na-osnove-ergonomicheskogo-analiza (дата обращения: 25.01.2025).

20. Оптимизация запросов базы данных на примере B2B сервиса для строителей, [Электронный ресурс]. ─ URL https://habr.com/ru/articles/461071/ (дата обращения 05.11.2024)

21. PostgresPRO. Документация.15.1. Как работают параллельно выполняемые запросы, [Электронный ресурс]. ─ URL https://postgrespro.ru/docs/postgresql/10/how-parallel-query-works (дата обращения 27.10.2024)

22. PostgreSQL documentation 64.1, [Электронный ресурс]. ─ URL https://www.postgresql.org/docs/current/btree.html (дата обращения 17.10.2024)

23. PostgresPro документация 14.1.1, [Электронный ресурс]. ─ URL https://postgrespro.ru/docs/postgrespro/9.5/using-explain (дата обращения 18.10.2024)

24. PostgreSQL documentation 65.6, [Электронный ресурс]. ─ URL https://www-PostgreSQL-org.translate.goog/docs/current/storage-page-layout.html?_x_tr_sl=auto&_x_tr_tl=ru&_x_tr_hl=ru (дата обращения 23.10.2024)

25. Блок компании Postgres Professional PostgreSQL*SQL*, [Электронный ресурс]. ─ URL https://habr.com/ru/companies/postgrespro/articles/330544/ (дата обращения 12.12.2024)

26. Индексы в PostgreSQL, [Электронный ресурс]. ─ URL https://habr.com/ru/companies/postgrespro/articles/326097/ (дата обращения 17.11.2024)

27. Почему оптимизатор запросов не анализирует содержимое буферного пула, [Электронный ресурс]. ─ URL https://habr.com/ru/companies/otus/articles/649573/ (дата обращения 03.01.2025)

28. MongoDB Documentation, [Электронный ресурс]. ─ URL https://www.MongoDB.com/docs/ (дата обращения 03.11.2024)

29. Адамо Тонете. Учебное пособие по шардингу MongoDB с лучшими практиками и рекомендациями по его включению, [Электронный ресурс]. ─ URL https://www-percona-com.translate.goog/blog/when-should-i-enable-MongoDB-sharding/?_x_tr_sl=en&_x_tr_tl=ru&_x_tr_hl=ru&_x_tr_pto=rq#:~:text=Auto%20sharding%20in%20MongoDB%20automatically,datasets%20and%20high%20user%20demands (дата обращения 10.12.2024)

30. Горячкин Б.С. Проектная эргономика автоматизированных систем обработки отображения информации и управления: монография / Б. С. Горячкин. Т. 1. ─ Москва: Спутник+, 2023-, 2023. — 496 с.: ил., табл., цв. ил..; ISBN 978-5-9973-6761-9.

31. От семантики к смыслу: будущее поисковых технологий. Консорциум СЭБ, Агенство Стратегических Инициатив, [Электронный ресурс]. ─ URL https://seb.e.lanbook.com/tpost/ilia2ndts1-ot-semantiki-k-smislu-buduschee-poiskovi (дата обращения 10.11.2024)

32. Построение базы знаний компании и поиска документов на LLM и RAG, [Электронный ресурс]. ─ URL https://habr.com/ru/companies/raft/articles/863888/ (дата обращения 10.01.2025)

33. Векторизация текста в NLP: от слов к числам, [Электронный ресурс]. ─ URL https://habr.com/ru/companies/skillfactory/articles/878112/ (дата обращения 30.01.2025)

34. BERT, [Электронный ресурс]. ─ URL https://huggingface.co/docs/transformers/en/model_doc/bert (дата обращения 31.01.2025)

35. Иванов И.И. Методы семантического поиска на основе эмбеддингов, [Электронный ресурс]. ─ URL https://elibrary.ru/item.asp?id=12345678 (дата обращения: 23.05.2025).

36. Петрова А.В. Анализ и улучшение методов оценки семантической близости текстов, [Электронный ресурс]. ─ URL https://elibrary.ru/item.asp?id=23456789 (дата обращения: 20.05.2025).

37. Смирнов К.В. Применение моделей глубокого обучения для задач семантического поиска, [Электронный ресурс]. – URL https://elibrary.ru/item.asp?id=34567890 (дата обращения: 20.05.2025).

38. Reimers N., Gurevych I. Sentence-BERT: Sentence Embeddings using Siamese BERT-Networks, [Электронный ресурс]. – URL https://arxiv.org/abs/1908.10084 (дата обращения: 22.05.2025).

39. Semantic Textual Similarity and Embeddings: A Survey, [Электронный ресурс]. – URL https://jair.org/index.php/jair/article/view/12345 (дата обращения: 23.05.2025).

ПРИЛОЖЕНИЕ А

ГРАФИЧЕСКИЕ МАТЕРИАЛЫ

А.1 – Мониторинг за нажатиями пользователя и отправляет данные на сервер с использованием яндекс метрики на сайте Библиоклуб.ру

А.2 – Мониторинг за нажатиями пользователя и отправляет данные на сервер с использованием яндекс метрики на сайте Znanium

А.3 – Подгрузка книг по одной странице на сайте Znanium

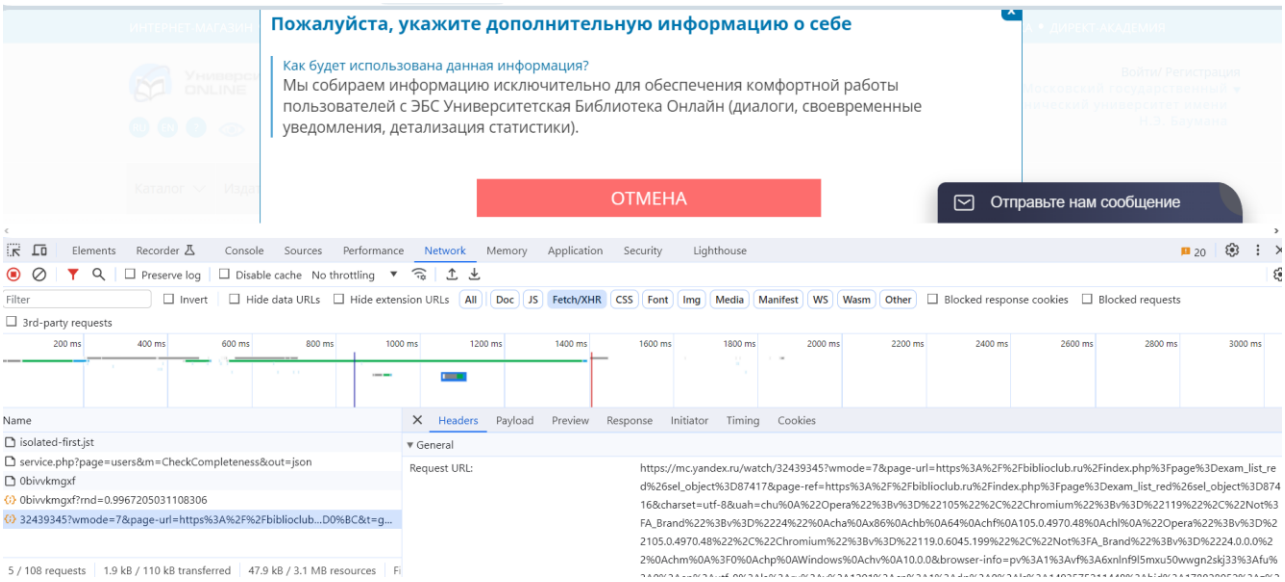
А.4 – Возможность ставить закладки и комментарии к страницам на сайте Znanium

А.5 – Расширенный поиск источника, включающий поиск по УДК, ISBN и т.п. на сайте Национальной электронной библиотеки

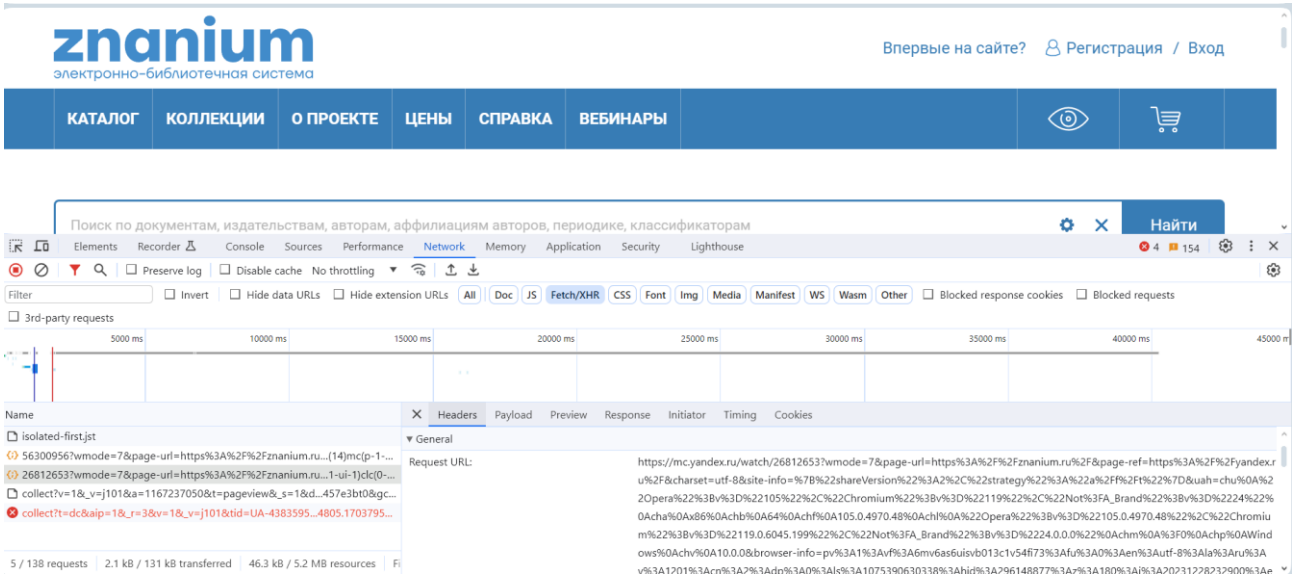
А.6 – Среди параметров поиска присутствует ISBN на сайте Национальной электронной библиотеки

А.7 – Платный доступ на сайте academia-moscow

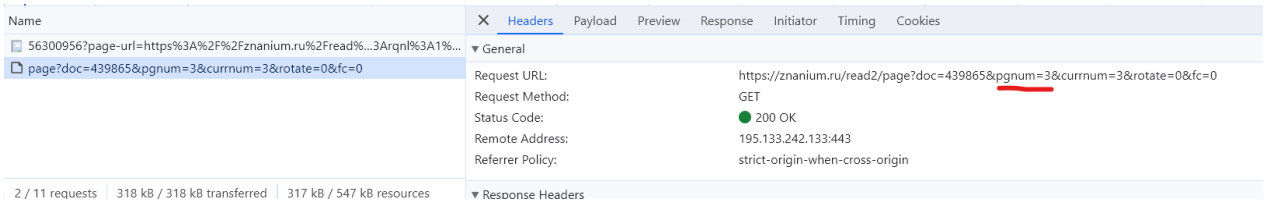
А.1 Мониторинг за нажатиями пользователя и отправляет данные на сервер с использованием яндекс метрики на сайте Библиоклуб.ру



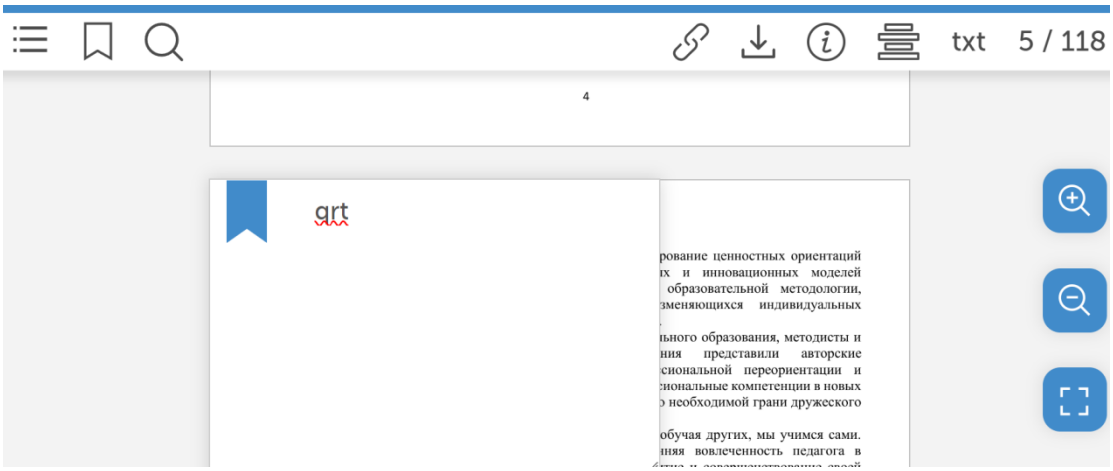
А.2 мониторинг за нажатиями пользователя и отправляет данные на сервер с использованием яндекс метрики на сайте Znanium



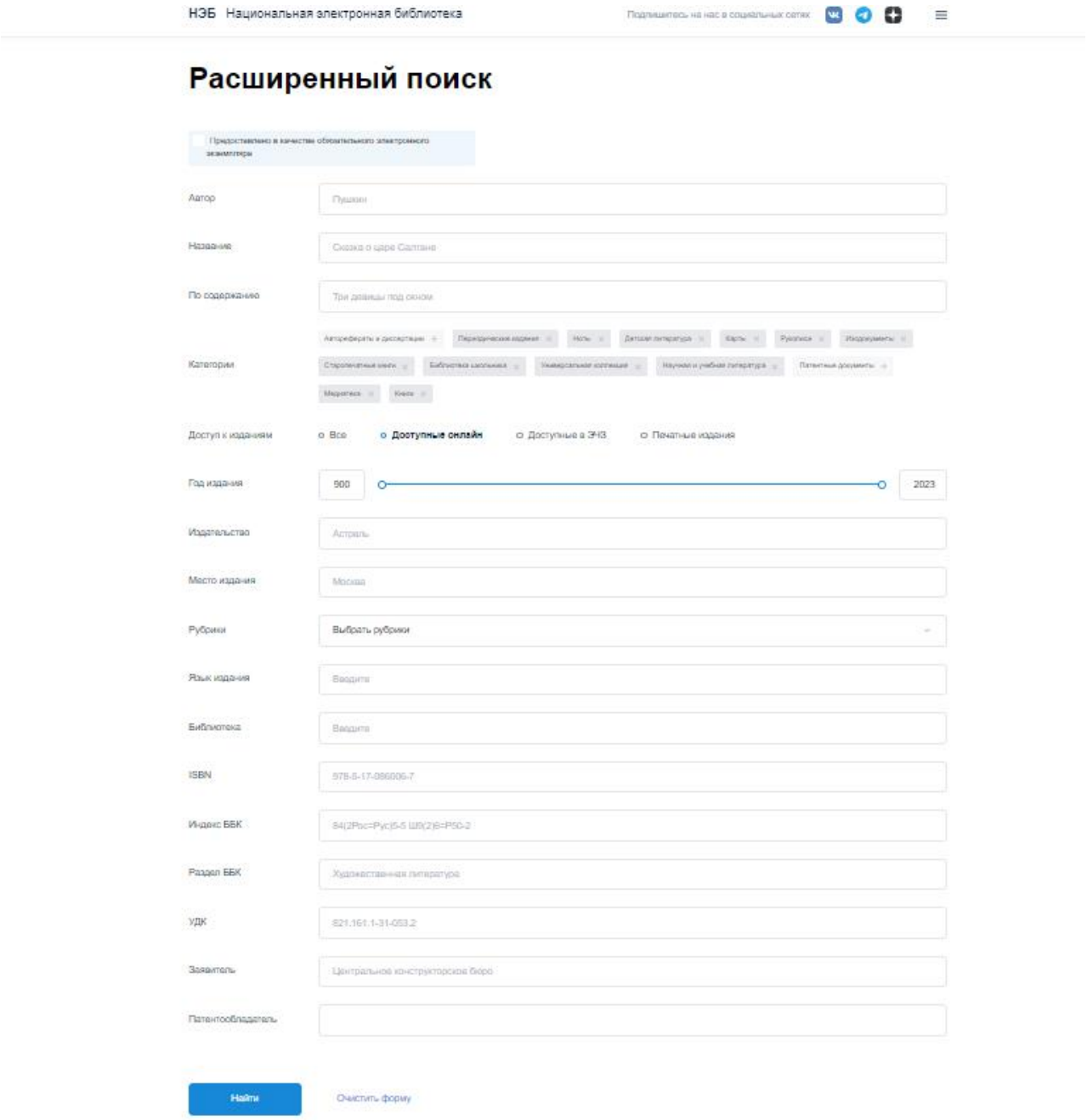
A.3 Подгрузка книг по одной странице на сайте Znanium



A.4 Возможность ставить закладки и комментарии к страницам на сайте Znanium

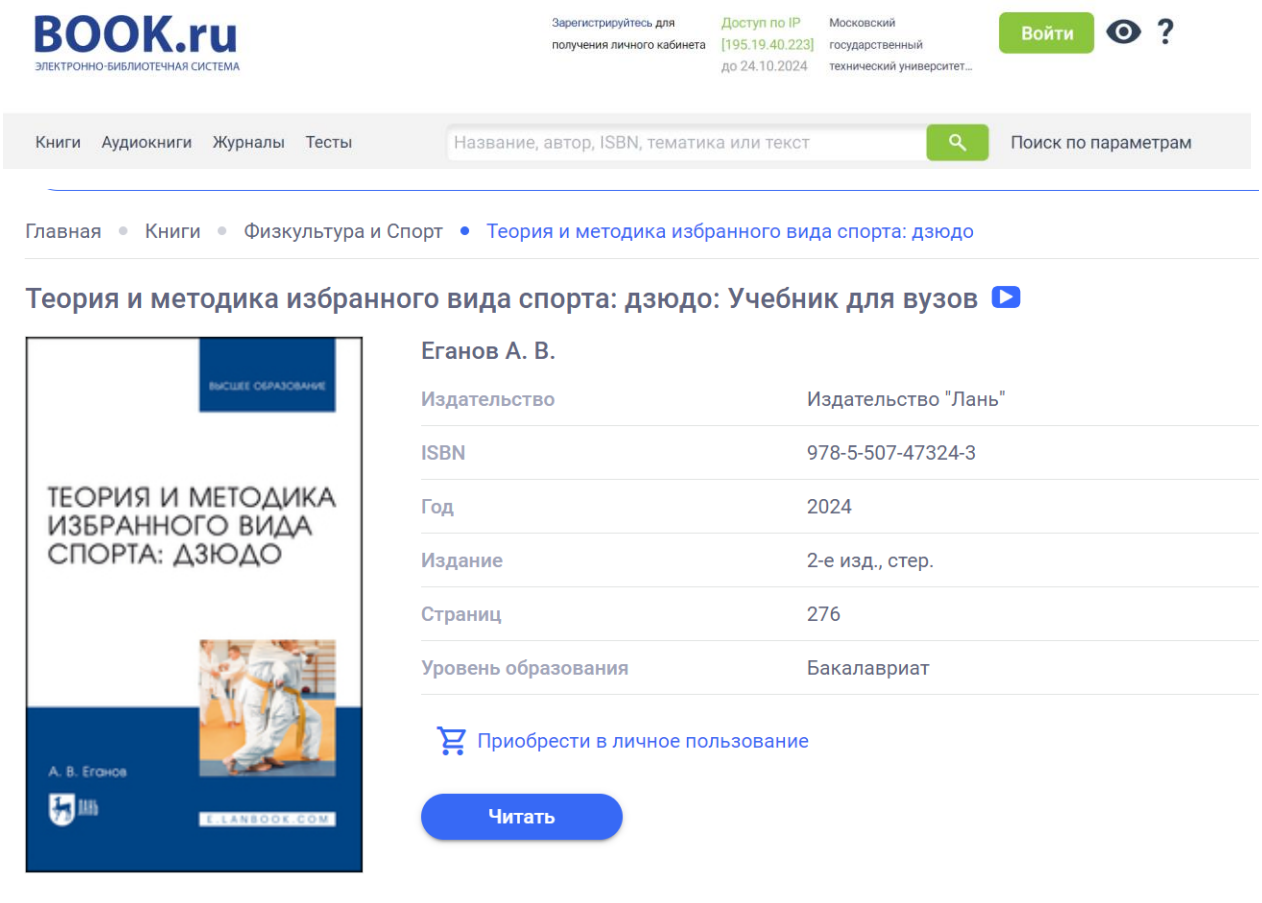


А.5 – Расширенный поиск источника, включающий поиск по УДК, ISBN и т.п. на сайте Национальной электронной библиотеки



;

А.6 – Среди параметров поиска присутствует ISBN на сайте Национальной электронной библиотеки



А.7 – Платный доступ на сайте academia-moscow

Главная / Электронная библиотека

- ОБЩЕОБРАЗОВАТЕЛЬНЫЕ ДИСЦИПЛИНЫ ДЛЯ СПО
- ПРОФЕССИОНАЛЬНОЕ ОБРАЗОВАНИЕ (ФГОС СПО)
- ОБЩЕЕ ОБРАЗОВАНИЕ
- ЭЛЕКТРОННАЯ ВЕРСИЯ КАТАЛОГА

Электронная библиотека

Единый сервис доступа к учебным материалам «Академии», который позволяет работать с цифровыми копиями печатных учебников и мультимедийными учебными изданиями в любое время при наличии подключения к сети Интернет.



Преимущества электронной библиотеки:

- Обеспечение учебными материалами всех студентов колледжа при выборе многопользовательской лицензии и/или выбор нужного количества доступов к изданиям.
- Возможность подключения и оформления договора за один день. Предоставляется пакет документов, подтверждающих обеспеченность образовательной организации учебной литературой.
- Удобный выбор изданий по укрупненной группе профессий и специальностей или по номеру ФГОС.
- Стоимость электронного издания в ЭБ значительно ниже стоимости печатного издания.

Обратите внимание на отличия в условиях приобретения изданий для физических и юридических лиц. Образовательные организации также могут заполнить форму запроса коммерческого предложения, представленную ниже, и получить обратную связь.

ЗАПРОС КОММЕРЧЕСКОГО ПРЕДЛОЖЕНИЯ

Название организации

Регион

Фамилия Имя Отчество

Должность

Запрос